

TEKO Schweizerische Fachschule

Software Engineering

Semesterarbeit

Parkhaus app

Autor:
Adrian Aeschlimann

Dozent:
Patrick Graber

Abgabedatum: 12.05.2026
Version 0.1

Abstract

Diese Semesterarbeit beschreibt die Erarbeitung eines Lastenhefts (siehe Anhang) sowie die Entwicklung eines funktionsfähigen Prototyps für eine neue Parkhaus-Verwaltungssoftware im Auftrag der fiktiven Firma EasyParking AG. Die bestehende Software des Unternehmens soll abgelöst werden. Da keine geeignete Standardlösung gefunden wurde, wird eine eigene Anwendung entwickelt.

Im ersten Teil der Arbeit werden die Anforderungen erhoben und in einem Lastenheft dokumentiert. Dabei werden funktionale, nicht-funktionale und organisatorische Anforderungen definiert. Die Analyse umfasst zwei Benutzerkategorien. Gelegenheitsnutzer, die ein Parkticket erhalten und vor der Ausfahrt abgerechnet werden, sowie Dauermieter mit fixem Parkplatz und monatlicher Miete.

Der zweite Teil umfasst die Konzeption und Umsetzung des Prototyps als Webanwendung. Als technische Grundlage dienen die Programmiersprache Elixir und das Web-Framework Phoenix mit LiveView. Der Prototyp bildet den vollständigen Parkierungsprozess ab. Von der Einfahrt über die Gebührenberechnung und Bezahlung bis zur Ausfahrt. Zusätzlich sind ein Admin Dashboard zur Verwaltung von Dauermietern sowie eine Statistikansicht mit Umsatzauswertungen implementiert.

Die Gebührenberechnung erfolgt auf Viertelstundenbasis mit konfigurierbaren Tarifen je nach Tageszeit, Wochentag und Feiertag (siehe Anhang). Bei einer Parkdauer von über 24 Stunden wird automatisch auf eine Tagespauschale umgestellt. Die Parkplatzzuweisung für Gelegenheitsnutzer basiert auf einem Algorithmus, der eine ausgeglichene Verteilung über die Stockwerke anstrebt.

Die Korrektheit der zentralen Geschäftslogik wird durch automatisierte Tests mit ExUnit sichergestellt.

Inhaltsverzeichnis

1. Abkürzungsverzeichnis	1
2. Projektinitialisierung	1
2.1. Ausgangslage	1
2.2. Situationsanalyse (IST-Zustand)	1
2.3. Rahmenbedingungen	1
2.4. Abgrenzung	1
2.5. Stakeholder-Analyse	2
2.6. Ziele	2
2.7. Grobe Anforderungen an das neue System	2
2.8. Lösungskonzept mit Varianten und Beurteilung	3
2.9. Projektmanagement	4
3. Konzept	7
3.1. Kontextdiagramm	7
3.2. Geschäftsprozessanalyse	7
3.3. Detailanforderungen an das neue System	9
3.4. Use-Case-Beschreibungen	10
3.5. Sequenzdiagramme	17
3.6. Modellierung der Klassen	22
3.7. Zustandsdiagramme	24
3.8. Modellierung der Datenbank	27
3.9. Systemarchitektur	28
3.10. Testkonzept	29
3.11. Einführungskonzept	30
3.12. Migrationskonzept	31
3.13. GUI-Design	31
4. Realisierung	32
4.1. Programmierumgebung / Programmierrichtlinien	32
4.2. Softwareaufbau	33
4.3. GUI-Implementierung	36
4.4. Datenbankimplementierung und -anbindung	39
4.5. Testprotokoll	41
5. Fazit / Lessons learned	48
6. Anhang	48
6.1. Parktarife	48
6.2. Lastenheft	48
Literaturverzeichnis	49
Tabellenverzeichnis	49
Abbildungsverzeichnis	50
Hilfsmittelverzeichnis	51
Glossar	51
Eigenständigkeitserklärung	52

1. Abkürzungsverzeichnis

Abkürzung	Bedeutung
BEAM	Bogdan's Erlang Abstract Machine
DSL	Domain Specific Language
UUID	Universally Unique Identifier

Tabelle 1: Abkürzungsverzeichnis

2. Projektinitialisierung

2.1. Ausgangslage

Die EasyParking AG betreibt mehrere Parkhäuser in verschiedenen Städten. Für den Betrieb dieser Parkhäuser wird aktuell eine Software verwendet, welche den Parkprozess unterstützt. Diese Software ist jedoch in die Jahre gekommen und soll spätestens bis Ende 2029 ersetzt werden.

Im Vorfeld wurde geprüft, ob eine bestehende Standardsoftware eingesetzt werden kann. Dazu wurde eine Marktanalyse durchgeführt. Die geprüften Produkte konnten die Anforderungen der EasyParking AG jedoch nicht ausreichend erfüllen. Aus diesem Grund wurde entschieden, eine eigene Software entwickeln zu lassen.

Die EasyParking AG verfügt über eine interne IT-Abteilung, hat jedoch keine eigenen Entwicklerkapazitäten für ein solches Projekt. Deshalb soll die Entwicklung der neuen Parkhaus-Software durch einen externen Anbieter erfolgen.

2.2. Situationsanalyse (IST-Zustand)

Die bestehende Software wird aktuell für den Betrieb der Parkhäuser verwendet und bildet den grundlegenden Parkprozess ab. Dazu gehört insbesondere der Ablauf von der Einfahrt eines Fahrzeugs bis zur Ausfahrt nach erfolgter Bezahlung.

Die bestehende Lösung erfüllt jedoch nicht mehr alle Anforderungen des Unternehmens. Anpassungen oder Erweiterungen sind nur schwer möglich. Deshalb soll die Software in den kommenden Jahren durch eine neue Lösung ersetzt werden.

2.3. Rahmenbedingungen

2.3.1. Prozessbezogene Rahmenbedingungen

Die Semesterarbeit wird als individuelles Projekt durchgeführt und erstreckt sich über einen Zeitraum von zwölf Wochen. Während dieser Zeit finden zwei Reviews statt, in denen der aktuelle Stand des Projekts überprüft wird.

Im Projekt sollen Methoden aus dem Softwareengineering angewendet werden. Dazu gehören insbesondere Planung, Analyse, Design und Dokumentation der entwickelten Lösung.

2.3.2. Produktbezogene Rahmenbedingungen

Die neue Software soll den Parkprozess innerhalb eines Parkhauses abbilden. Dazu gehören insbesondere Einfahrt, Parkieren, Berechnung der Parkgebühren und Ausfahrt.

Das System soll mehrere Parkhäuser verwalten können. Die Struktur eines Parkhauses, insbesondere die Anzahl Stockwerke sowie die Anzahl Parkplätze pro Stockwerk, muss im System definiert werden können.

Externe Systeme wie das Zahlungssystem und die Buchhaltung bleiben bestehen und werden über Schnittstellen angebunden.

2.4. Abgrenzung

Nicht Bestandteil dieses Projekts sind externe Systeme wie das Zahlungssystem oder die Buchhaltung. Diese Systeme werden lediglich über Schnittstellen angebunden.

Die physische Steuerung von Schranken oder Ticketautomaten gehört ebenfalls nicht zum Projektumfang. Für den Prototyp wird die Interaktion mit diesen Komponenten am Bildschirm simuliert.

2.5. Stakeholder-Analyse

Stakeholder	Interesse am Projekt
Geschäftsleitung	Entscheidung über Einführung der neuen Software
IT-Leitung	Technische Bewertung der Lösung
Betreiber der Parkhäuser	Zuverlässiger Betrieb der Parkhäuser
Softwareentwickler	Umsetzung der Anforderungen
Kunden der Parkhäuser	Einfache Nutzung des Parksystems

Tabelle 2: Stakeholderanalyse

2.6. Ziele

Das Hauptziel des Projekts ist die Entwicklung eines Prototyps für eine neue Parkhaus-Software. Diese Software soll den Parkprozess vollständig unterstützen und mehrere Parkhäuser verwalten können.

Zusätzlich soll eine strukturierte Projektdokumentation erstellt werden, welche Analyse, Design und Umsetzung der Lösung beschreibt.

2.7. Grobe Anforderungen an das neue System

Die neue Software soll den Parkprozess von der Einfahrt bis zur Ausfahrt unterstützen. Dabei müssen zwei Arten von Nutzern berücksichtigt werden: Gelegenheitsnutzer und Dauermieter.

Beim Einfahren eines Gelegenheitsnutzers wird ein Parkticket erstellt. Dieses enthält Informationen über die Einfahrtszeit und den zugewiesenen Parkplatz.

Dauermieter erhalten einen festen Parkplatz und können das Parkhaus mithilfe eines persönlichen Codes betreten.

Die Software muss ausserdem Parkgebühren berechnen können. Die Berechnung basiert auf der Parkdauer sowie auf definierten Tarifen, die je nach Tageszeit variieren können.

Zusätzlich soll das System Auswertungen über die Nutzung und die Umsätze eines Parkhauses erstellen können.

2.8. Lösungskonzept mit Varianten und Beurteilung

2.8.1. Varianten

Kriterium	Desktop-Anwendung	Webanwendung	Bewertung
Installation	Lokale Installation notwendig	Keine Installation, läuft im Browser	Web Vorteil
Zugriff	Nur auf installiertem Gerät	Von überall mit Browser zugänglich	Web Vorteil
Hardware-Anbindung	Direkter Zugriff möglich	Nur über Schnittstellen möglich	Desktop Vorteil
Wartung	Updates pro Gerät notwendig	Zentrale Updates	Web Vorteil
Entwicklungs-komplexität	Einfacher für Prototyp	Höherer Aufwand	Desktop Vorteil
Skalierbarkeit	Begrenzt	Gut skalierbar	Web Vorteil
Offline-Fähigkeit	Gut möglich	Eingeschränkt	Desktop Vorteil
Geeignet für Prototyp	Sehr gut geeignet	Eher aufwendig	Desktop Vorteil

Tabelle 3: Variantenvergleich

Für die Umsetzung der Parkhaus-Software wurden zwei grundlegende Varianten betrachtet. Einerseits eine klassische Desktop-Anwendung, andererseits eine webbasierte Anwendung.

Die Desktop-Anwendung wird lokal installiert und ausgeführt. Sie kann direkt auf lokale Ressourcen zugreifen und ist einfacher umzusetzen, insbesondere für einen Prototyp. Der Nachteil liegt in der Verteilung und Wartung, da Updates auf jedem System einzeln durchgeführt werden müssen.

Die Webanwendung wird zentral betrieben und über einen Browser genutzt. Dadurch ist sie von verschiedenen Geräten aus zugänglich und einfacher zu warten. Allerdings ist die Entwicklung aufwendiger und der direkte Zugriff auf Hardware ist eingeschränkt.

2.8.1.1. Webframework

Kriterium	Elixir / Phoenix	Express.js	Bewertung
Nebenläufigkeit	Eingebaut (BEAM)	Begrenzt (Single-threaded)	Phoenix Vorteil
Lernkurve	Hoch	Niedrig	Express Vorteil
Ökosystem	Klein, spezialisiert	Gross, weit verbreitet	Express Vorteil
Echtzeit-UI	LiveView nativ	Zusätzliche Bibliotheken nötig	Phoenix Vorteil
Vorwissen	Keine Erfahrung	Grundkenntnisse vorhanden	Express Vorteil
Stabilität	BEAM garantiert Fehlertoleranz	Abhängig von Implementierung	Phoenix Vorteil

Tabelle 4: Variantenvergleich Webframework

Für Express.js sind Grundkenntnisse vorhanden, was die Einarbeitung vereinfacht hätte. Elixir und Phoenix bieten jedoch eine bessere Grundlage für eine robuste, nebenläufige Anwendung. Ausserdem war das Kennenlernen einer neuen Technologie ein explizites Lernziel. Deshalb wurde Elixir/Phoenix gewählt.

2.8.1.2. Vorgehensmodell

Kriterium	Wasserfall	Scrum	Bewertung
Planbarkeit	Hoch	Gering	Wasserfall Vorteil
Flexibilität	Gering	Hoch	Scrum Vorteil
Teamgrösse	Geeignet für Einzelperson	Für Teams ausgelegt	Wasserfall Vorteil
Meilensteine	Klar definiert	Sprint-basiert	Wasserfall Vorteil
Dokumentation	Klar strukturiert	Iterativ	Wasserfall Vorteil
Overhead	Gering	Hoch (Meetings, Rollen)	Wasserfall Vorteil

Tabelle 5: Variantenvergleich Vorgehensmodell

Scrum ist für Teams mit iterativer Entwicklung ausgelegt. Da das Projekt von einer einzelnen Person durchgeführt wird und die Meilensteine durch die Semesterarbeit vorgegeben sind, ist das Wasserfallmodell besser geeignet. Der geringere organisatorische Overhead kommt einem Einzelprojekt zugute.

2.8.2. Machbarkeitsbeurteilung

Beide Varianten (Desktop und Web) sind grundsätzlich realisierbar. Die Webanwendung bietet Vorteile im späteren Betrieb, insbesondere bei Wartung und Zugriff. Für den Umfang dieser Semesterarbeit ist sie jedoch mit höherem Aufwand verbunden. Die Desktop-Anwendung ist einfacher umzusetzen und erlaubt eine schnelle Entwicklung eines funktionalen Prototyps.

Beide Webframeworks sind für die Umsetzung geeignet. Express.js erlaubt dank vorhandener Grundkenntnisse einen schnelleren Einstieg. Elixir und Phoenix sind technisch überlegen, erfordern jedoch eine intensive Einarbeitungszeit.

Sowohl Wasserfall als auch Scrum sind grundsätzlich anwendbar. Für ein Einzelprojekt mit vorgegebenen Meilensteinen bietet das Wasserfallmodell klare Vorteile, da der organisatorische Overhead von Scrum ohne Team kaum gerechtfertigt ist.

2.8.3. Variantenentscheid

Für dieses Projekt wird eine Webanwendung gewählt. Diese Entscheidung basiert darauf, dass eine funktionierende Grundlage aufgebaut wird, welche in der Zukunft einfach erweitert werden kann. Ausserdem ermöglicht eine Webanwendung eine einfachere Demonstration des Produktes, ohne dass der Benutzer den Prototyp aufsetzen muss.

Als Webframework wird Elixir/Phoenix eingesetzt. Trotz höherer Lernkurve bietet die Plattform bessere Stabilität und eine native Lösung für Echtzeit-Weboberflächen über LiveView. Das Kennenlernen einer neuen Technologie war zudem ein explizites Lernziel.

Als Vorgehensmodell wird das Wasserfallmodell gewählt. Die Meilensteine sind durch die Semesterarbeit vorgegeben und das Projekt wird von einer einzelnen Person durchgeführt, weshalb der Mehraufwand von Scrum nicht gerechtfertigt ist.

2.9. Projektmanagement

2.9.1. Projektplanung

Für die Umsetzung des Projekts wird als Vorgehensmodell das Wasserfallmodell gewählt. Das Projekt wird in klar getrennte Phasen unterteilt.

Die gewählten Phasen orientieren sich an der Vorlage der Semesterarbeit und bestehen aus Initialisierung, Konzept, Realisierung und Dokumentation.

Der Zeitplan ist in Form eines Gantt-Charts dargestellt. Die Planung basiert auf den vorgegebenen Terminen und teilt die Arbeit in mehrere zusammenhängende Aufgaben auf. Während der Realisierung und Dokumentation gibt es bewusst Überschneidungen, um die Dokumentation parallel zur Umsetzung zu erstellen.

Die Planung ist bewusst einfach gehalten und fokussiert sich auf die wichtigsten Meilensteine sowie auf eine sinnvolle zeitliche Aufteilung der Arbeit.

Die Ressourcenplanung ist überschaubar, da das Projekt im Rahmen einer Semesterarbeit von einer einzelnen Person durchgeführt wird. Es stehen keine weiteren Entwickler oder zusätzliche Ressourcen zur Verfügung. Dies wurde bei der Planung berücksichtigt, indem der Umfang auf einen realistischen Prototyp begrenzt wurde.

2.9.2. Zeitplan

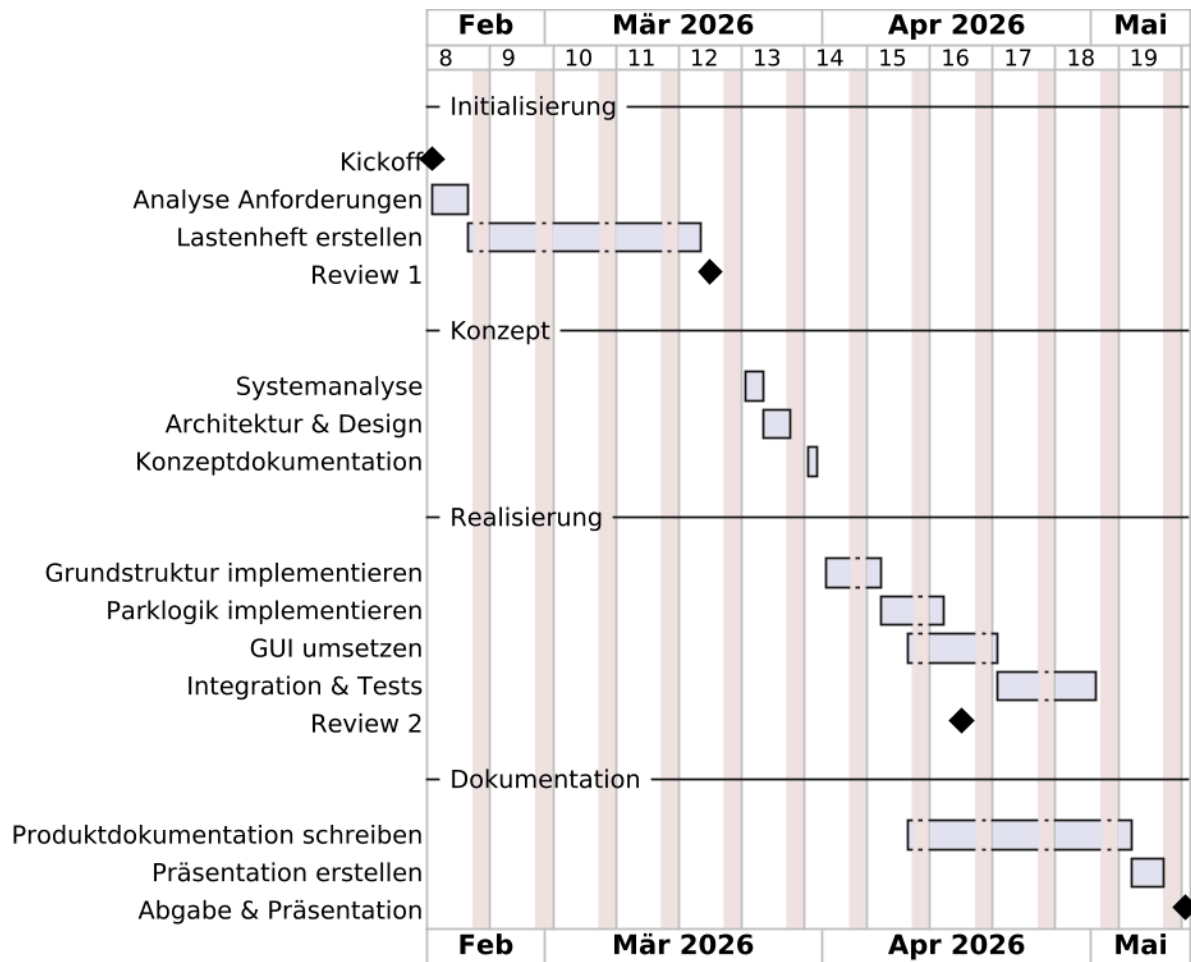


Abbildung 1: Zeitplan

2.9.3. Risikoanalyse

Risiko	Beschreibung	Massnahme
Zeitmangel	Mehraufwand durch neue Technologie	Kernfunktionen priorisieren, Buffer einplanen
Komplexität	Unbekannte Konzepte erhöhen Komplexität	Einfach halten, schrittweise umsetzen
Technologierisiko	Probleme durch fehlende Erfahrung	Gezielt einarbeiten, Alternativen bereithalten

Tabelle 6: Risikoanalyse

2.9.4. Qualitätsmanagement

Die Qualität der Lösung wird durch regelmässige Überprüfung der Anforderungen sowie durch Reviews während des Projekts sichergestellt. Zusätzlich wird die Software getestet, um die korrekte Funktion der wichtigsten Abläufe zu prüfen.

2.9.5. Konfigurationsmanagement

Der Quellcode sowie die Projektdokumentation werden versioniert gespeichert. Änderungen werden nachvollziehbar dokumentiert, sodass jederzeit der aktuelle Stand des Projekts ersichtlich ist.

Wie der Code sowohl auch die Dokumentation sind in einem Github-Repository gespeichert:

<https://github.com/Adyrem/parking>

3. Konzept

3.1. Kontextdiagramm

Das Kontextdiagramm zeigt die Beziehung zwischen dem Parkhaus-System und externen Systemen. Dazu gehören insbesondere das Zahlungssystem sowie die Buchhaltung. Diese Systeme tauschen Daten mit der Parkhaus-Software über Schnittstellen aus.

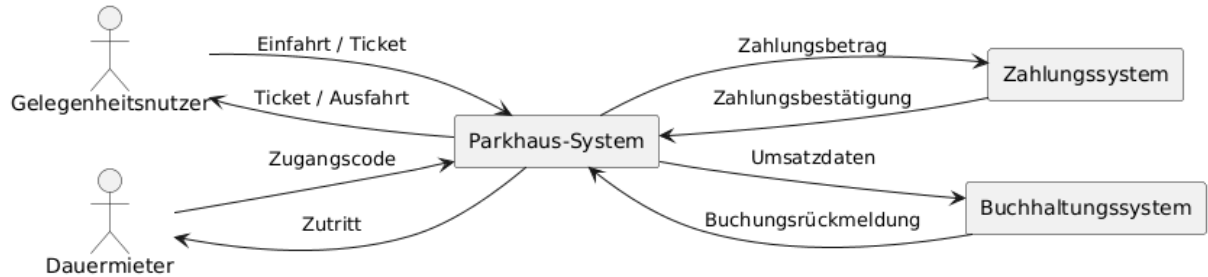


Abbildung 2: Kontextdiagramm

3.2. Geschäftsprozessanalyse

Der zentrale Geschäftsprozess ist das Parkieren eines Fahrzeugs. Dieser Prozess beginnt mit der Einfahrt in das Parkhaus und endet mit der Ausfahrt.

Beim Einfahren erhält ein Gelegenheitsnutzer ein Parkticket. Während der Parkdauer wird ein Parkplatz belegt. Vor der Ausfahrt wird das Ticket bezahlt und danach kann das Fahrzeug das Parkhaus verlassen.

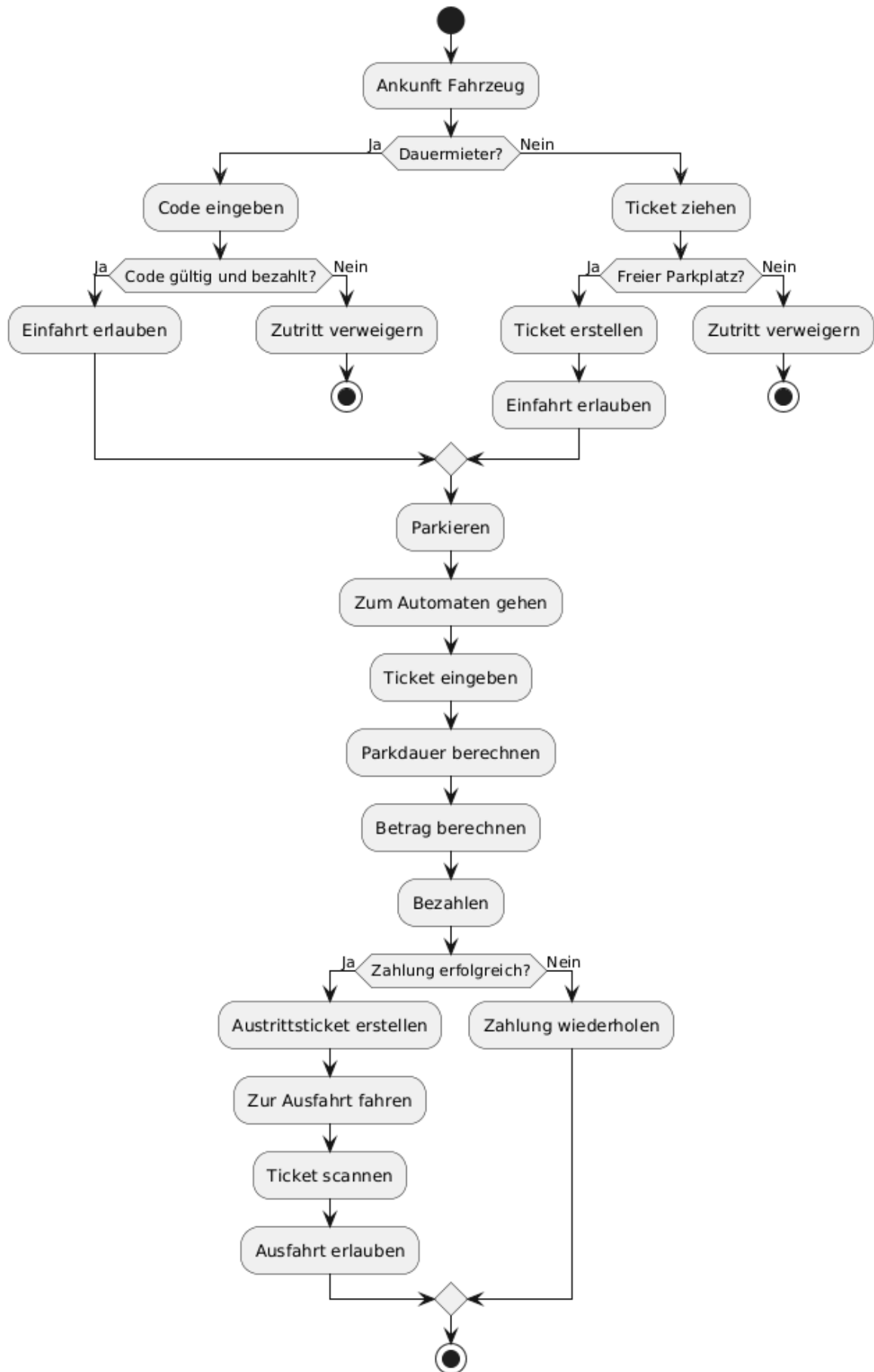


Abbildung 3: Geschäftsprozess

3.3. Detailanforderungen an das neue System

3.3.1. Funktionale Anforderungen

ID	Anforderung	Beschreibung
F-01	Parkhäuser verwalten	Mehrere Parkhäuser im System verwalten
F-02	Parkhaus konfigurieren	Stockwerke und Parkplätze definieren
F-03	Benutzerkategorien	Gelegenheitsnutzer und Dauermieter unterscheiden
F-04	Ticket erstellen	Ticket bei Einfahrt generieren
F-05	Einfahrt registrieren	Datum und Zeit speichern
F-06	Code prüfen	Zugangscode von Dauermietern prüfen
F-07	Parkplatz zuweisen	Automatische Zuweisung eines Parkplatzes
F-08	Fixe Parkplätze	Dauermietern feste Plätze zuweisen
F-09	Parkdauer berechnen	Dauer zwischen Ein- und Ausfahrt bestimmen
F-10	Gebühren berechnen	Kosten basierend auf Tarif berechnen
F-11	Tarife verwalten	Tarife nach Tageszeit, Wochentag und Feiertagen anwenden. Konkrete Tarife siehe Anhang
F-12	Viertelstundenabrechnung	Abrechnung in 15-Minuten-Schritten. Der Tarif zu Beginn einer Viertelstunde gilt für die gesamte Viertelstunde
F-13	Tagespauschale	Automatische Tagespauschale von CHF 35.00 pro Tag ab 24 Stunden Parkdauer. Angebrochene Tage werden vollständig verrechnet
F-14	Sperrlogik	Dauermieter bei Nichtzahlung sperren
F-15	Ein-/Ausfahrten protokollieren	Alle Bewegungen speichern
F-16	Parkplätze anzeigen	Freie und belegte Plätze darstellen
F-17	Statistiken erstellen	Nutzungsdaten auswerten
F-18	Monatsumsatz berechnen	Umsatz pro Monat berechnen
F-19	Jahresumsatz berechnen	Umsatz pro Jahr berechnen
F-20	Dauermieter verwalten	Dauermieter erstellen und verwalten
F-21	Austrittsticket erstellen	Nach erfolgreicher Zahlung ein Austrittsticket generieren und am Bildschirm anzeigen

Tabelle 7: Funktionale Anforderungen

3.3.2. Nicht-funktionale Anforderungen

ID	Anforderung	Beschreibung
NF-01	Zuverlässigkeit	System funktioniert stabil und fehlerfrei
NF-02	Performance	Schnelle Reaktion bei Einfahrt und Berechnung
NF-03	Benutzbarkeit	Einfache und verständliche Bedienung
NF-04	Korrektheit	Gebühren und Zeiten werden korrekt berechnet
NF-05	Verfügbarkeit	System ist jederzeit nutzbar
NF-06	Sicherheit	Zugriffe und Daten sind geschützt
NF-07	Wartbarkeit	System kann einfach angepasst werden
NF-08	Erweiterbarkeit	Neue Funktionen können ergänzt werden

Tabelle 8: Nicht-funktionale Anforderungen

3.3.3. Organisatorische Anforderungen

ID	Anforderung	Beschreibung
O-01	Einzelprojekt	Projekt wird von einer Person durchgeführt
O-02	Dokumentation	Alle Ergebnisse werden dokumentiert
O-03	Reviews	Teilnahme an zwei Reviews
O-04	Termine	Einhaltung der vorgegebenen Termine
O-05	Prototyp	Erstellung eines funktionsfähigen Prototyps
O-06	Nachvollziehbarkeit	Arbeitsschritte müssen verständlich sein

Tabelle 9: Organisatorische Anforderungen

3.4. Use-Case-Beschreibungen

Name	Einfahrt Gelegenheitsnutzer
Nummer	UC-01
Kurzbeschreibung	Ein Nutzer fährt ins Parkhaus ein und erhält ein Ticket
Stakeholder	Gelegenheitsnutzer
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	Ticketautomat, Schranke
Vorbedingungen	Parkplätze verfügbar
Nachbedingungen	Ticket erstellt, Einfahrt erfolgt
Typischer Ablauf	Knopf drücken → Ticket wird erstellt → Schranke öffnet
Alternative Abläufe	Kein Parkplatz → Einfahrt verweigert
Kritikalität	Hoch
Verknüpfungen	UC-03, UC-04
Funktionale Anforderungen	F-04 Ticket erstellen, F-05 Einfahrt registrieren, F-07 Parkplatz zuweisen
Nicht-funktionale Anforderungen	NF-01 Zuverlässigkeit, NF-02 Performance

Tabelle 10: Use Case UC-01

Name	Einfahrt Dauermieter
Nummer	UC-02
Kurzbeschreibung	Dauermieter fährt mit Code ein
Stakeholder	Dauermieter
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	Schranke
Vorbedingungen	Code vorhanden
Nachbedingungen	Einfahrt registriert
Typischer Ablauf	Code eingeben → Prüfung → Schranke öffnet
Alternative Abläufe	Code ungültig → Zutritt verweigert
Kritikalität	Hoch
Verknüpfungen	UC-04
Funktionale Anforderungen	F-06 Code prüfen, F-05 Einfahrt registrieren
Nicht-funktionale Anforderungen	NF-06 Sicherheit, NF-05 Verfügbarkeit

Tabelle 11: Use Case UC-02

Name	Bezahlung Parkticket
Nummer	UC-03
Kurzbeschreibung	Berechnung und Bezahlung der Parkgebühr
Stakeholder	Gelegenheitsnutzer
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	Zahlungssystem
Vorbedingungen	Ticket vorhanden
Nachbedingungen	Ticket entwertet, Austrittsticket erstellt
Typischer Ablauf	Ticket scannen → Betrag berechnen → bezahlen → Austrittsticket erstellen
Alternative Abläufe	Zahlung fehlgeschlagen → erneut versuchen
Kritikalität	Sehr hoch
Verknüpfungen	UC-01, UC-04
Funktionale Anforderungen	F-09 Parkdauer berechnen, F-10 Gebühren berechnen, F-11 Tarife verwalten, F-12 Viertelstundenabrechnung, F-13 Tagespauschale, F-21 Austrittsticket erstellen
Nicht-funktionale Anforderungen	NF-04 Korrektheit, NF-02 Performance

Tabelle 12: Use Case UC-03

Name	Ausfahrt
Nummer	UC-04
Kurzbeschreibung	Fahrzeug verlässt das Parkhaus
Stakeholder	Alle Nutzer
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	Schranke
Vorbedingungen	Gültiges Austrittsticket (Gelegenheitsnutzer) oder gültiger Code (Dauermieter)
Nachbedingungen	Ausfahrt registriert
Typischer Ablauf	Austrittsticket scannen bzw. Code eingeben → Ausfahrtszeit registrieren → Schranke öffnet
Alternative Abläufe	Ticket ungültig → Ausfahrt verweigert
Kritikalität	Sehr hoch
Verknüpfungen	UC-01, UC-02, UC-03
Funktionale Anforderungen	F-05 Einfahrt registrieren, F-15 Ein-/Ausfahrten protokollieren, F-21 Austrittsticket erstellen
Nicht-funktionale Anforderungen	NF-01 Zuverlässigkeit

Tabelle 13: Use Case UC-04

Name	Parkplätze anzeigen
Nummer	UC-05
Kurzbeschreibung	Anzeige freier und belegter Parkplätze
Stakeholder	Betreiber
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	-
Vorbedingungen	Parkhaus konfiguriert
Nachbedingungen	Aktuelle Belegung sichtbar
Typischer Ablauf	System zeigt Status pro Parkplatz
Alternative Abläufe	-
Kritikalität	Mittel
Verknüpfungen	-
Funktionale Anforderungen	F-16 Parkplätze anzeigen
Nicht-funktionale Anforderungen	NF-03 Benutzbarkeit

Tabelle 14: Use Case UC-05

Name	Parkhaus konfigurieren
Nummer	UC-06
Kurzbeschreibung	Struktur eines Parkhauses definieren
Stakeholder	Betreiber
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	-
Vorbedingungen	System gestartet
Nachbedingungen	Parkhaus ist konfiguriert
Typischer Ablauf	Stockwerke und Parkplätze definieren
Alternative Abläufe	Ungültige Eingaben → Korrektur
Kritikalität	Hoch
Verknüpfungen	UC-01, UC-05
Funktionale Anforderungen	F-01 Parkhäuser verwalten, F-02 Parkhaus konfigurieren
Nicht-funktionale Anforderungen	NF-03 Benutzbarkeit

Tabelle 15: Use Case UC-06

Name	Parkplatz zuweisen
Nummer	UC-07
Kurzbeschreibung	Automatische Zuweisung eines Parkplatzes
Stakeholder	System
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	-
Vorbedingungen	Freie Parkplätze vorhanden
Nachbedingungen	Parkplatz zugewiesen
Typischer Ablauf	System wählt Parkplatz
Alternative Abläufe	Keine Plätze → Abbruch
Kritikalität	Hoch
Verknüpfungen	UC-01
Funktionale Anforderungen	F-07 Parkplatz zuweisen
Nicht-funktionale Anforderungen	NF-02 Performance

Tabelle 16: Use Case UC-07

Name	Parkgebühr berechnen
Nummer	UC-08
Kurzbeschreibung	Berechnung der Parkkosten
Stakeholder	System
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	-
Vorbedingungen	Einfahrtszeit vorhanden
Nachbedingungen	Betrag berechnet
Typischer Ablauf	Parkdauer berechnen → Wochentag/Feiertag prüfen → geltenden Tarif anwenden
Alternative Abläufe	>24h → Tagespauschale; Wochenende oder Feiertag → Wochenend-/Feiertagstarif
Kritikalität	Sehr hoch
Verknüpfungen	UC-03
Funktionale Anforderungen	F-09 Parkdauer berechnen, F-10 Gebühren berechnen, F-11 Tarife verwalten
Nicht-funktionale Anforderungen	NF-04 Korrektheit

Tabelle 17: Use Case UC-08

Name	Dauermieter verwalten
Nummer	UC-09
Kurzbeschreibung	Verwaltung von Dauermietern
Stakeholder	Betreiber
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	-
Vorbedingungen	System verfügbar
Nachbedingungen	Daten gespeichert
Typischer Ablauf	Daten erfassen/ändern
Alternative Abläufe	-
Kritikalität	Mittel
Verknüpfungen	UC-02
Funktionale Anforderungen	F-03 Benutzerkategorien, F-20 Dauermieter verwalten
Nicht-funktionale Anforderungen	NF-06 Sicherheit

Tabelle 18: Use Case UC-09

Name	Sperrstatus prüfen
Nummer	UC-10
Kurzbeschreibung	Prüfung ob Dauermieter gesperrt ist
Stakeholder	System
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	-
Vorbedingungen	Dauermieter vorhanden
Nachbedingungen	Status bestimmt
Typischer Ablauf	Zahlungsstatus prüfen
Alternative Abläufe	Nicht bezahlt → Sperre
Kritikalität	Hoch
Verknüpfungen	UC-02
Funktionale Anforderungen	F-14 Sperrlogik
Nicht-funktionale Anforderungen	NF-01 Zuverlässigkeit

Tabelle 19: Use Case UC-10

Name	Statistiken erstellen
Nummer	UC-11
Kurzbeschreibung	Auswertung von Nutzungsdaten
Stakeholder	Betreiber
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	-
Vorbedingungen	Daten vorhanden
Nachbedingungen	Statistik erstellt
Typischer Ablauf	Zeitraum wählen → Daten anzeigen
Alternative Abläufe	-
Kritikalität	Mittel
Verknüpfungen	-
Funktionale Anforderungen	F-17 Statistiken erstellen
Nicht-funktionale Anforderungen	NF-02 Performance

Tabelle 20: Use Case UC-11

Name	Umsatz berechnen
Nummer	UC-12
Kurzbeschreibung	Berechnung von Monats- und Jahresumsätzen
Stakeholder	Betreiber
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	Buchhaltungssystem
Vorbedingungen	Daten vorhanden
Nachbedingungen	Umsatz berechnet
Typischer Ablauf	Zeitraum wählen → Berechnung
Alternative Abläufe	-
Kritikalität	Hoch
Verknüpfungen	UC-11
Funktionale Anforderungen	F-18 Monatsumsatz berechnen, F-19 Jahresumsatz berechnen
Nicht-funktionale Anforderungen	NF-04 Korrektheit

Tabelle 21: Use Case UC-12

Name	Austrittsticket erstellen
Nummer	UC-13
Kurzbeschreibung	Generierung und Anzeige des Austrittstickets nach erfolgreicher Zahlung
Stakeholder	Gelegenheitsnutzer
Fachverantwortliche Person	Adrian Aeschlimann
Referenzen	-
Vorbedingungen	Parkgebühr erfolgreich bezahlt
Nachbedingungen	Austrittsticket am Bildschirm angezeigt; Ticket als bezahlt markiert
Typischer Ablauf	Zahlung abgeschlossen → Austrittsticket generieren → Austrittsticket am Bildschirm anzeigen
Alternative Abläufe	-
Kritikalität	Sehr hoch
Verknüpfungen	UC-03, UC-04
Funktionale Anforderungen	F-21 Austrittsticket erstellen
Nicht-funktionale Anforderungen	NF-01 Zuverlässigkeit, NF-03 Benutzbarkeit

Tabelle 22: Use Case UC-13

3.5. Sequenzdiagramme

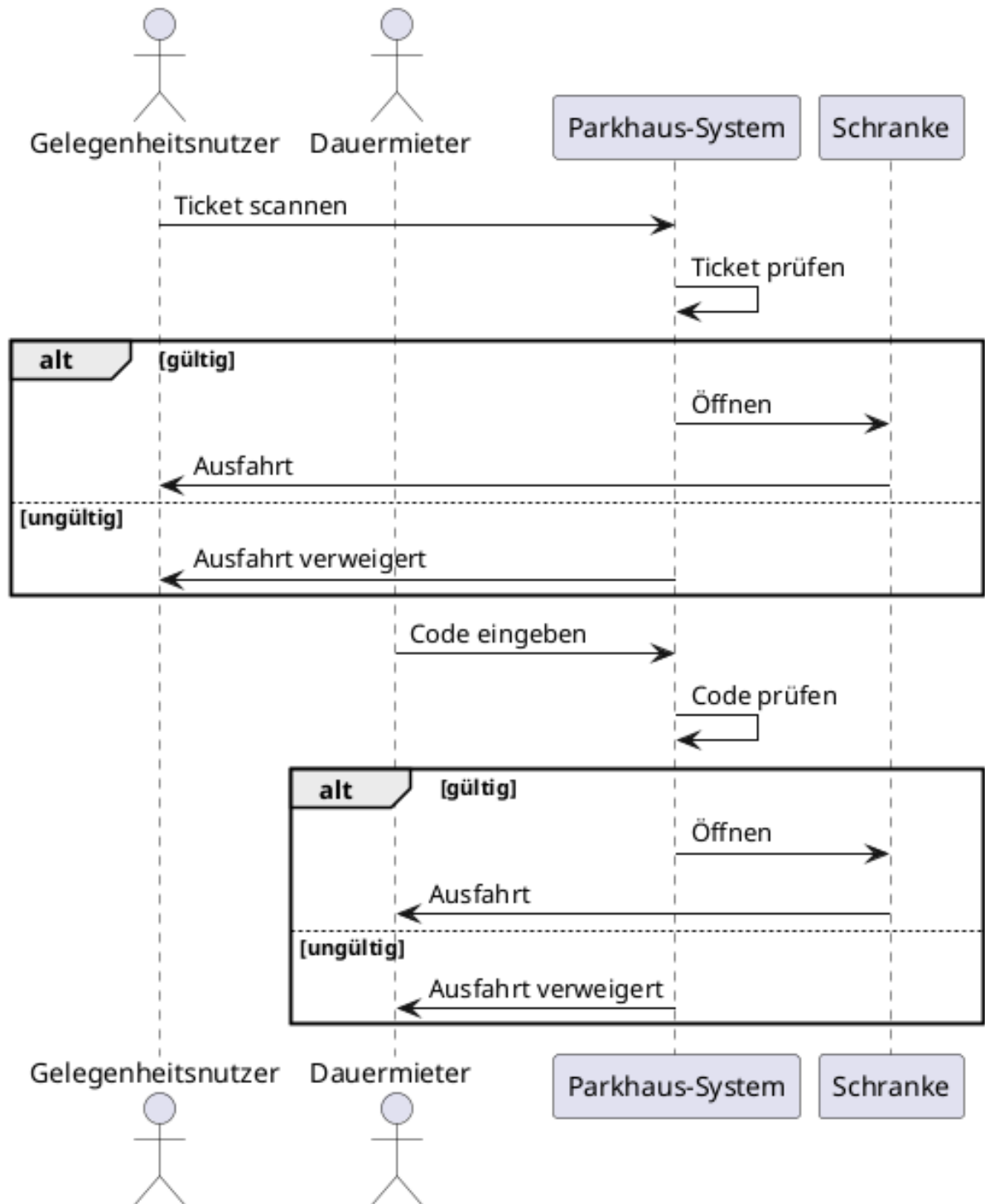


Abbildung 4: Ausfahrt

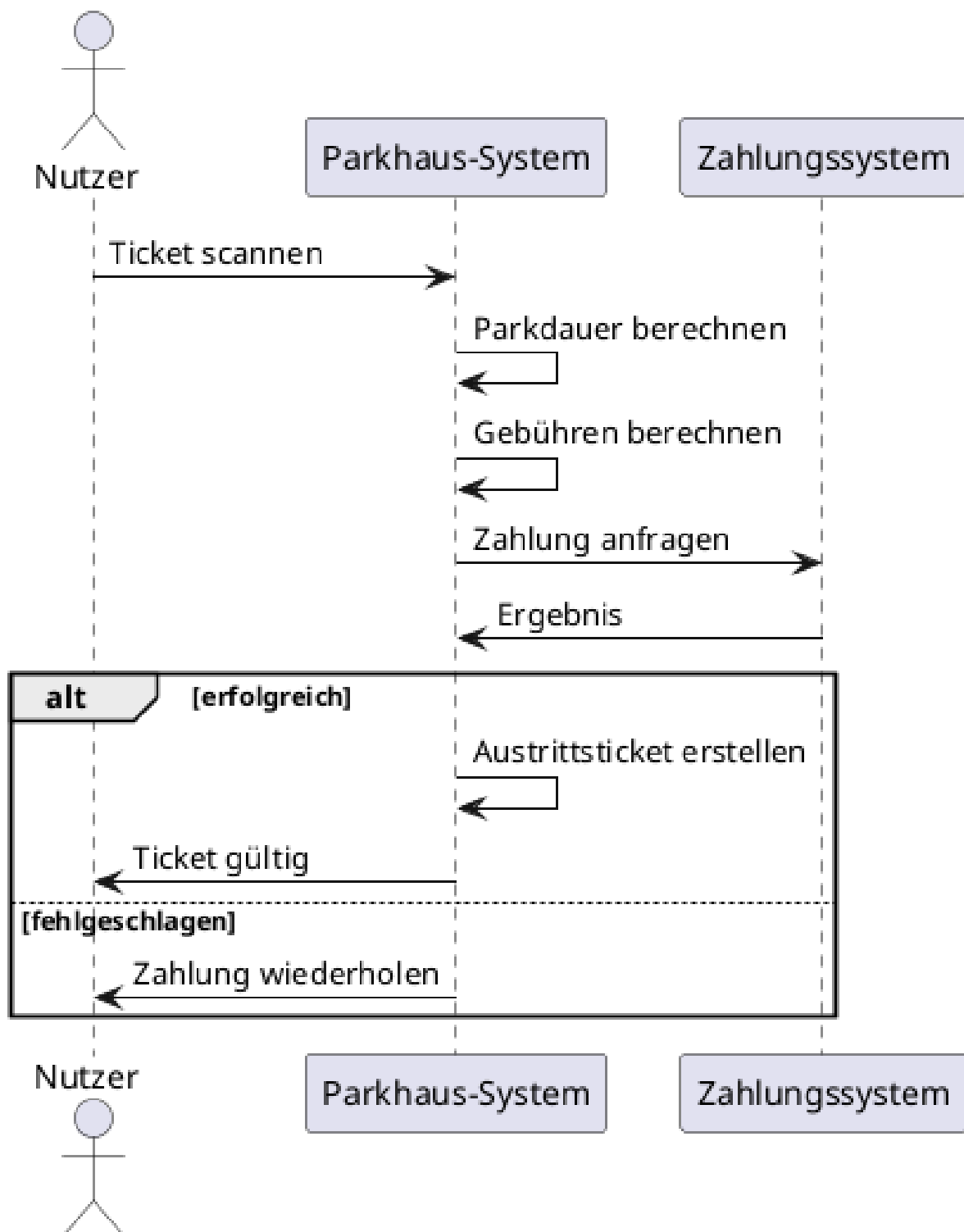


Abbildung 5: Bezahlung

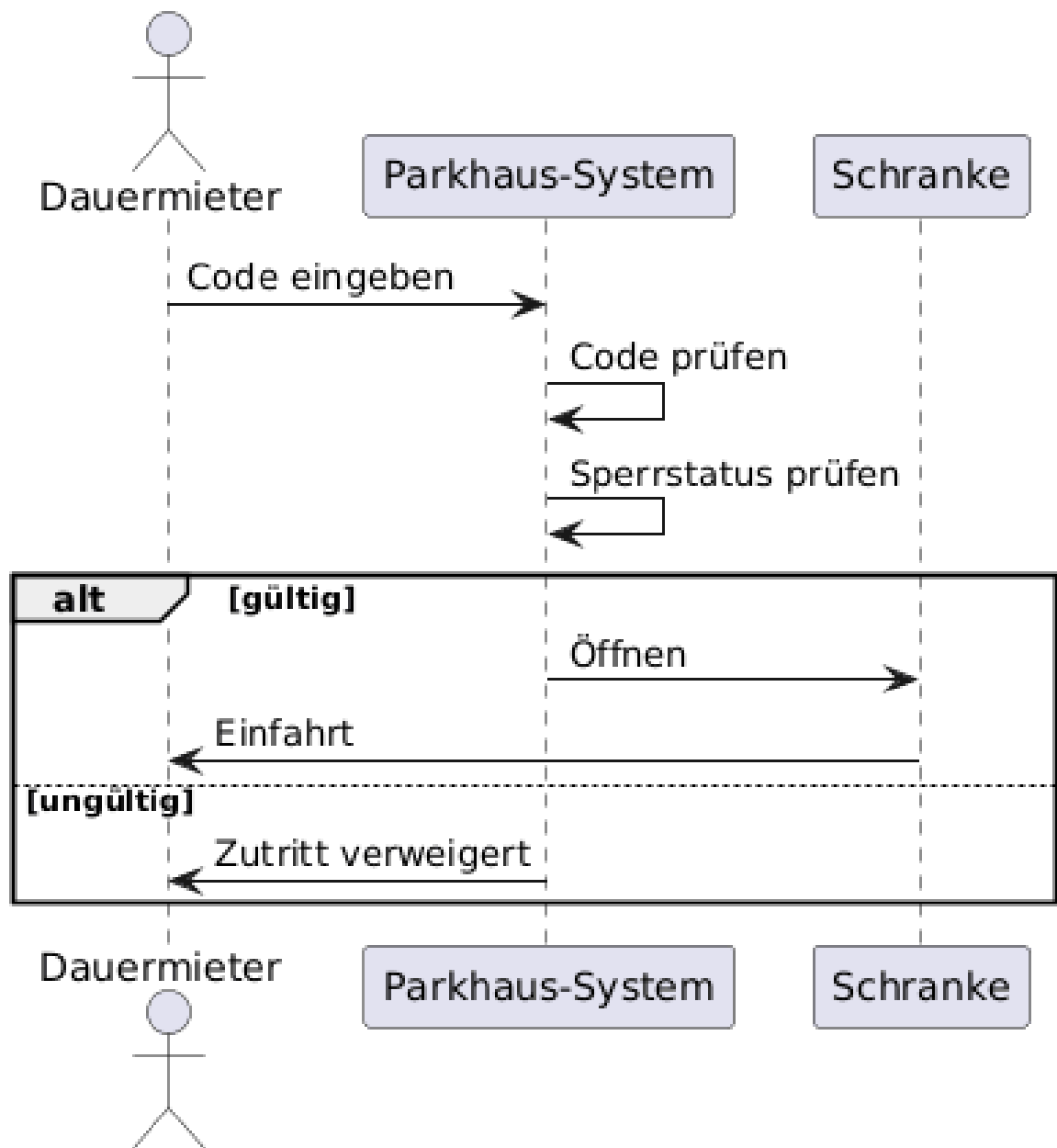


Abbildung 6: Einfahrt Dauermieter

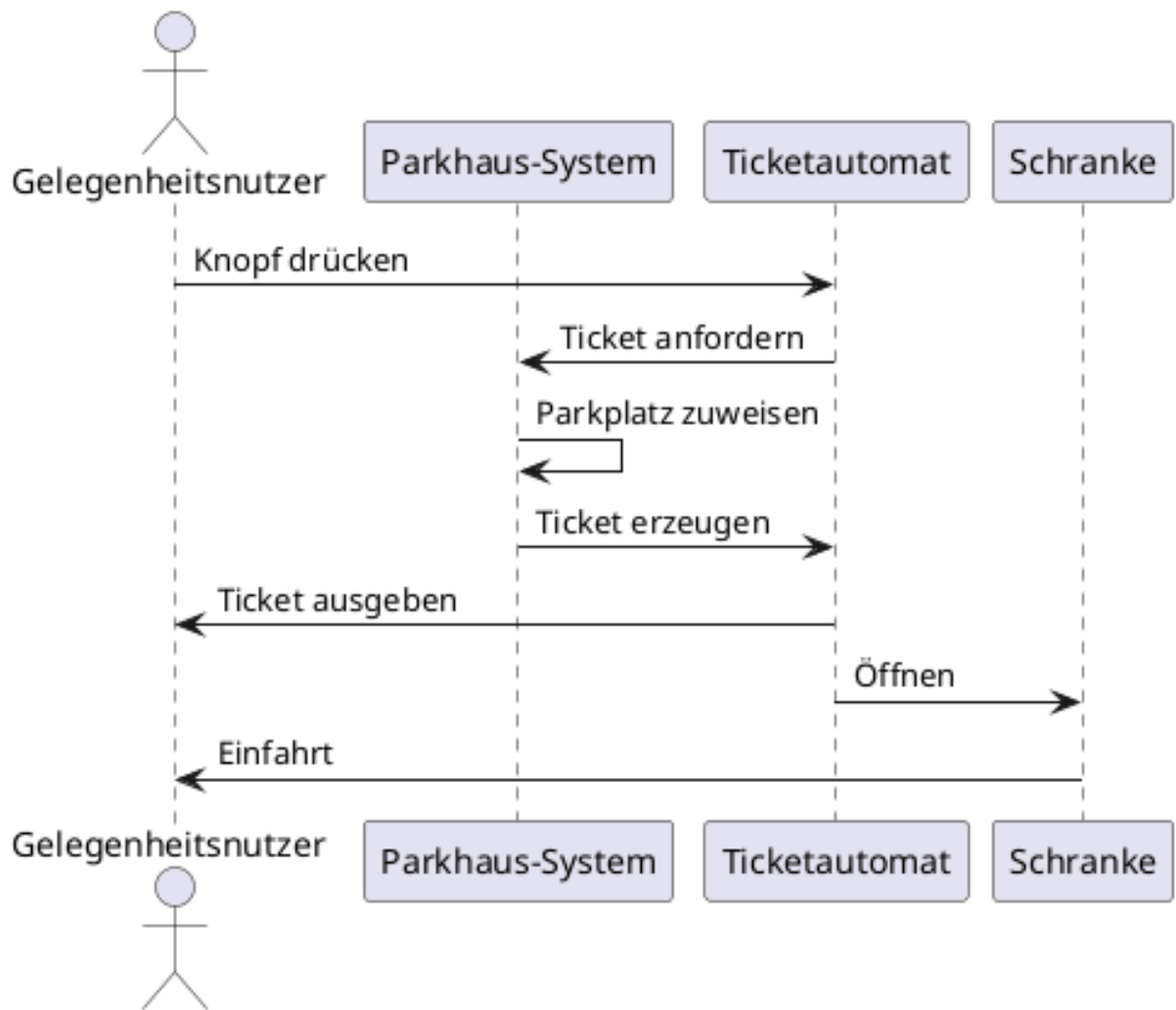


Abbildung 7: Einfahrt Gelegenheitsnutzer

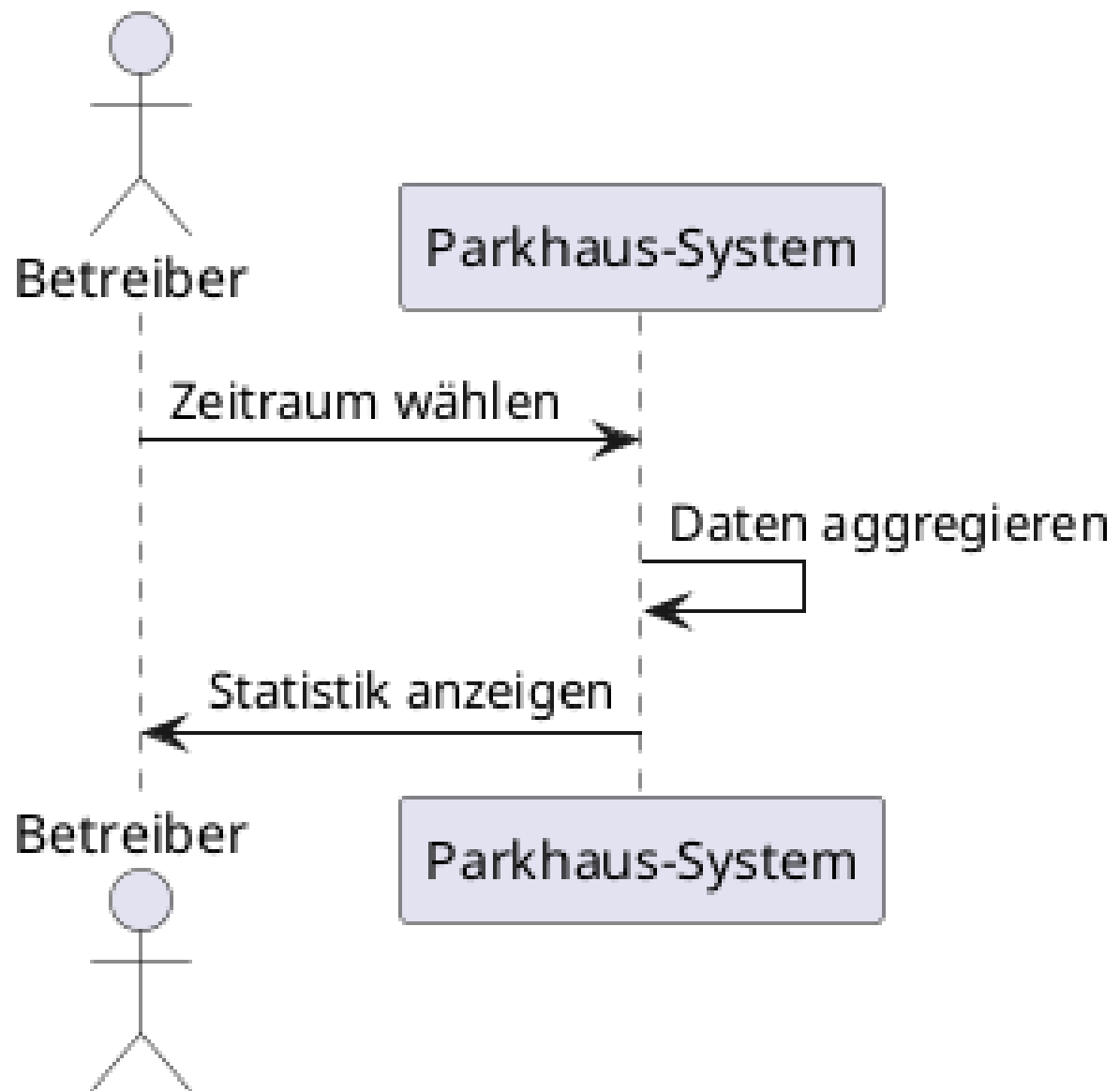


Abbildung 8: Statistiken (Umsatz)

3.6. Modellierung der Klassen

3.6.1. Klassendiagramm

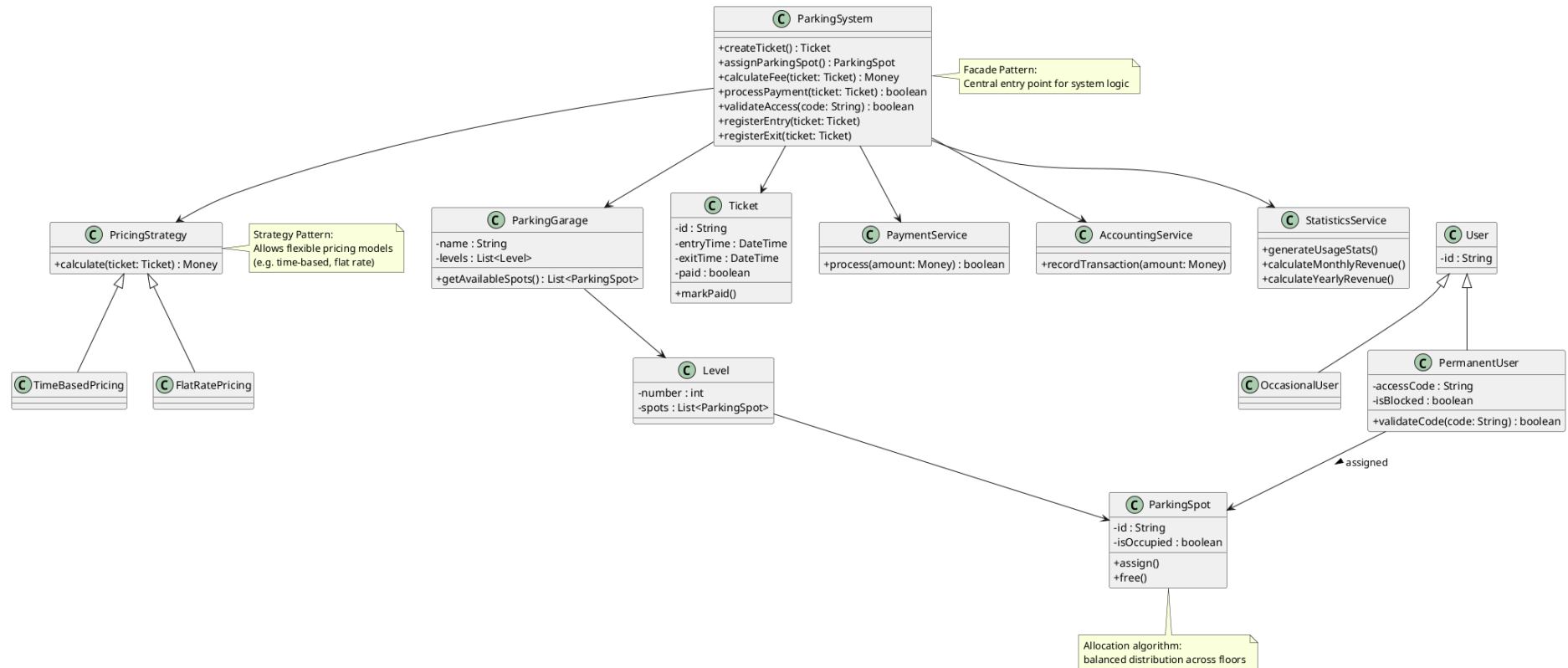


Abbildung 9: Klassendiagramm

3.6.2. Beschreibung der Fachklassen

ParkingSystem

Die Klasse ParkingSystem bildet die zentrale Steuerung des gesamten Systems. Sie übernimmt die Koordination aller Abläufe wie Einfahrt, Ausfahrt, Ticketverarbeitung, Parkplatzzuweisung, Gebührenberechnung sowie Zahlungsabwicklung. Dabei fungiert sie als zentrale Schnittstelle zwischen den einzelnen Komponenten und kapselt die Geschäftslogik. Die Klasse ruft je nach Anwendungsfall spezialisierte Services auf, beispielsweise für Zahlungsabwicklung oder Statistik. Durch diese Bündelung wird die Komplexität reduziert und die Interaktion mit dem System vereinfacht.

ParkingGarage

Die Klasse ParkingGarage repräsentiert ein vollständiges Parkhaus. Sie enthält mehrere Ebenen und bietet Funktionen zur Verwaltung und Abfrage von Parkplätzen. Eine zentrale Aufgabe ist die Ermittlung freier Parkplätze, die als Grundlage für die automatische Zuweisung dient. Die Struktur erlaubt eine Erweiterung auf mehrere Parkhäuser innerhalb desselben Systems.

Level

Die Klasse Level dient der Strukturierung eines Parkhauses in einzelne Ebenen. Sie verwaltet eine Menge von Parkplätzen und ermöglicht so eine klare Organisation innerhalb des Parkhauses.

ParkingSpot

Die Klasse ParkingSpot repräsentiert einen einzelnen Parkplatz. Sie enthält Informationen über den aktuellen Belegungsstatus sowie eine eindeutige Identifikation. Zentrale Methoden sind das Belegen und Freigeben eines Parkplatzes. Diese Klasse ist ein zentraler Bestandteil der Parkplatzlogik, da sie direkt von der Zuweisung und der Anzeige beeinflusst wird.

Ticket

Die Klasse Ticket enthält alle relevanten Daten eines Parkvorgangs. Dazu gehören insbesondere Einfahrtszeit, Ausfahrtszeit sowie der Zahlungsstatus. Sie bildet die Grundlage für mehrere Kernfunktionen des Systems, darunter die Berechnung der Parkdauer und der Gebühren sowie die Validierung bei der Ausfahrt. Das Ticket durchläuft verschiedene Zustände, etwa „aktiv“ oder „bezahlt“, wodurch der Lebenszyklus eines Parkvorgangs abgebildet wird.

User

Abstrakte Basisklasse für alle Benutzer des Systems. Enthält grundlegende Identifikationsmerkmale.

OccasionalUser

Repräsentiert einen Gelegenheitsnutzer, der das Parkhaus einmalig oder unregelmässig nutzt. Diese Benutzer erhalten bei der Einfahrt ein Ticket, welches später zur Bezahlung und Ausfahrt benötigt wird. Es erfolgt keine dauerhafte Speicherung von Nutzerdaten.

PermanentUser

Repräsentiert einen Dauermieter mit wiederkehrender Nutzung. Die Klasse verfügt über zusätzliche Attribute wie Zugangscode und Sperrstatus. Ein zentraler Bestandteil ist die Prüfung des Zugangscode sowie die Überprüfung, ob der Benutzer aufgrund offener Zahlungen gesperrt ist. Es wird ein fester Parkplatz zugewiesen, wodurch sich das Verhalten bei der Parkplatzvergabe unterscheidet.

PaymentService

Die Klasse PaymentService kapselt die gesamte Zahlungslogik und dient als Schnittstelle zu einem externen Zahlungssystem. Sie übernimmt die Verarbeitung von Zahlungen und liefert das Ergebnis an das System zurück. Durch diese Trennung bleibt die Kernlogik unabhängig von konkreten Zahlungsanbietern. Im Rahmen dieses Projektes wird hier ein externes System simuliert.

AccountingService

Zuständig für die Übergabe von Finanzdaten an ein externes Buchhaltungssystem. Erfasst Umsätze und stellt diese für Auswertungen bereit. Auch hier wird das externe System nur simuliert.

PricingStrategy

Die abstrakte Klasse PricingStrategy definiert die Schnittstelle zur Berechnung der Parkgebühren. Sie ermöglicht es, unterschiedliche Berechnungslogiken unabhängig voneinander zu implementieren. Dieses Konzept erlaubt eine flexible Anpassung von Tarifen oder erstellen einer neuen Berechnungslogik, ohne die zentrale Systemlogik zu verändern.

TimeBasedPricing

Diese Klasse implementiert eine zeitbasierte Gebührenberechnung. Die Kosten werden anhand der Parkdauer und definierter Tarife bestimmt, beispielsweise mit Abrechnung in Viertelstunden. Zusätzlich können Regeln wie unterschiedliche Tarife je nach Tageszeit oder Wochentag berücksichtigt werden.

FlatRatePricing

Die Klasse FlatRatePricing bildet eine eigenständige Pauschalpreisberechnung ab, die unabhängig von der Tageszeit eine einheitliche Tagespauschale anwendet.

StatisticsService

Die Klasse StatisticsService verarbeitet die im System gespeicherten Daten und stellt Auswertungen zur Verfügung. Dazu gehören z.B. Belegungsstatistiken sowie die Berechnung von Monats- und Jahresumsätzen. Die Berechnungen basieren auf protokollierten Ein-/Ausfahrten sowie den zugehörigen Zahlungen. Dadurch können betriebliche Kennzahlen einfach analysiert werden.

3.7. Zustandsdiagramme

Das Zustandsdiagramm des ParkingSpot zeigt den Belegungsstatus eines Parkplatzes. Ein Parkplatz ist initial frei und kann durch das System reserviert werden. Sobald ein Fahrzeug den Platz belegt, wechselt der Zustand zu belegt. Nach der Ausfahrt wird der Parkplatz wieder freigegeben und steht erneut zur Verfügung.

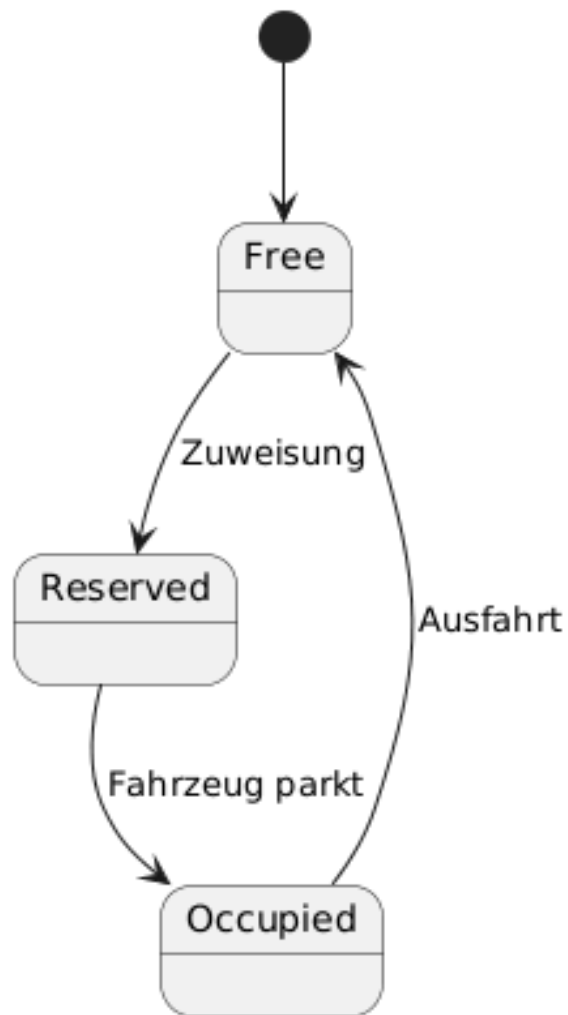


Abbildung 10: Parkplatz Zustandsdiagramm

Das Zustandsdiagramm des Ticket beschreibt den gesamten Lebenszyklus eines Parkvorgangs. Nach der Erstellung bei der Einfahrt wechselt das Ticket in den aktiven Zustand, in dem die Parkdauer läuft. Nach der Bezahlung wird es als bezahlt markiert und kann für die Ausfahrt verwendet werden. Erfolgt die Ausfahrt, wird das Ticket abgeschlossen.



Abbildung 11: Ticket Zustandsdiagramm

3.8. Modellierung der Datenbank

3.8.1. ERD

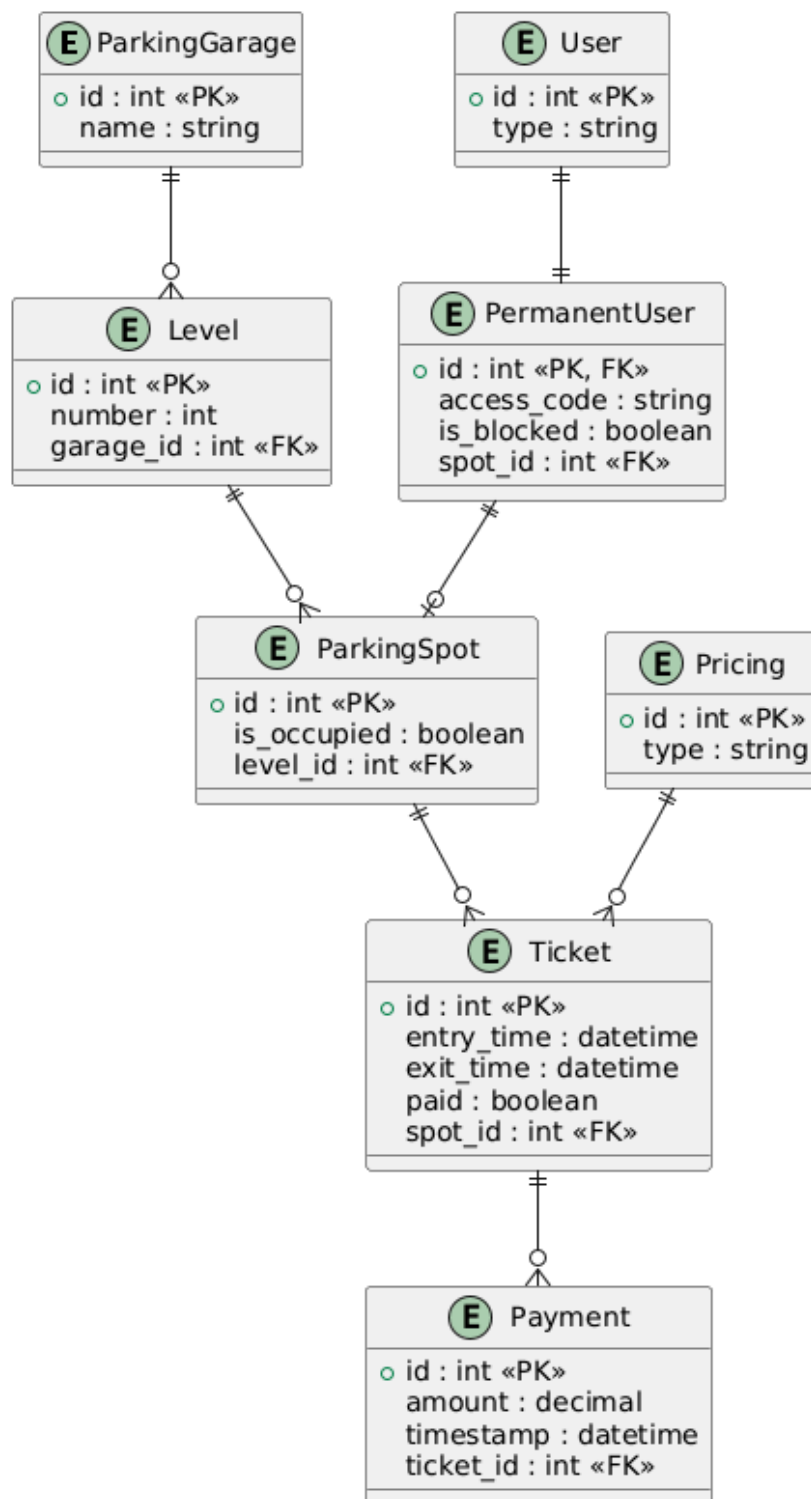


Abbildung 12: ERD

3.8.2. Beschreibung der Fachentitäten, Beziehungen und der referenziellen Integritätsbedingungen

ParkingGarage

Die Entität ParkingGarage repräsentiert ein Parkhaus und bildet die oberste strukturelle Einheit. Ein Parkhaus besteht aus mehreren Ebenen. Jede zugehörige Ebene muss eindeutig einem existierenden

Parkhaus zugeordnet sein, wodurch sichergestellt wird, dass keine Ebene ohne gültige Referenz existiert.

Level

Die Entität Level beschreibt eine Ebene innerhalb eines Parkhauses. Jede Ebene gehört genau zu einem Parkhaus und enthält mehrere Parkplätze. Die referenzielle Integrität verlangt, dass jede Ebene eine gültige Referenz auf ein bestehendes Parkhaus besitzt. Beim Löschen eines Parkhauses muss berücksichtigt werden, dass zugehörige Ebenen ebenfalls behandelt werden.

ParkingSpot

Die Entität ParkingSpot stellt einen einzelnen Parkplatz dar. Jeder Parkplatz gehört genau zu einer Ebene und kann im Zeitverlauf mehreren Parkvorgängen zugeordnet sein. Die Integritätsbedingungen stellen sicher, dass jeder Parkplatz eine gültige Ebene referenziert. Zudem darf ein Parkplatz nur dann gelöscht werden, wenn keine aktiven Referenzen, beispielsweise durch laufende Parkvorgänge, bestehen.

Ticket

Die Entität Ticket repräsentiert einen Parkvorgang. Sie speichert Einfahrts- und Ausfahrtszeit sowie den Zahlungsstatus und ist einem Parkplatz zugeordnet. Jedes Ticket muss auf einen existierenden Parkplatz verweisen. Zusätzlich kann ein Ticket mit mehreren Zahlungsvorgängen verknüpft sein. Die referenzielle Integrität stellt sicher, dass keine Zahlungen ohne gültiges Ticket existieren und dass ein Ticket nicht gelöscht werden kann, solange abhängige Zahlungen vorhanden sind.

User

Die Entität User bildet die Basis für Benutzer im System. Sie enthält grundlegende Identifikationsmerkmale und dient als Ausgangspunkt für Spezialisierungen. Die Integrität stellt sicher, dass spezialisierte Benutzer nur existieren können, wenn ein entsprechender Basiseintrag vorhanden ist.

PermanentUser

Die Entität PermanentUser erweitert die Basisklasse User um zusätzliche Informationen wie Zugangscode und Sperrstatus. Jeder Eintrag in dieser Entität muss auf einen existierenden Benutzer verweisen. Optional kann ein fester Parkplatz zugewiesen werden. In diesem Fall muss der referenzierte Parkplatz existieren. Wird ein Parkplatz gelöscht, muss geprüft werden, ob er einem Dauermieter zugewiesen ist.

Payment

Die Entität Payment speichert einzelne Zahlungsvorgänge. Jeder Zahlungseintrag ist genau einem Ticket zugeordnet. Die referenzielle Integrität stellt sicher, dass keine Zahlung ohne gültiges Ticket existieren kann. Beim Löschen eines Tickets müssen zugehörige Zahlungen ebenfalls berücksichtigt oder entfernt werden.

Pricing

Die Entität Pricing repräsentiert Tarifmodelle zur Gebührenberechnung. Ein Tarif kann mehreren Tickets zugeordnet sein. Die Integrität verlangt, dass jedes Ticket auf ein gültiges Tarifmodell verweist. Änderungen an Tarifmodellen müssen so erfolgen, dass bestehende Ticketdaten konsistent bleiben.

3.9. Systemarchitektur

Die Systemarchitektur des Parkhaus-Systems ist als Web-Anwendung aufgebaut. Ziel ist eine klare Trennung der Verantwortlichkeiten sowie eine einfache und nachvollziehbare Struktur, die sich gut erweitern lässt.

Das System folgt einer schichtenbasierten Architektur, bestehend aus Präsentationsschicht, Anwendungsschicht und Datenhaltungsschicht. Die Präsentationsschicht wird als Web-Frontend

umgesetzt und läuft im Browser. Sie dient ausschliesslich der Interaktion mit dem Benutzer und enthält keine Geschäftslogik. Benutzeraktionen werden an die Anwendungsschicht weitergeleitet.

Die Anwendungsschicht bildet den Kern des Systems und enthält die gesamte Geschäftslogik. Hier werden alle zentralen Abläufe wie Einfahrt, Ausfahrt, Ticketverarbeitung, Parkplatzzuweisung und Gebührenberechnung umgesetzt. Die Klasse ParkingSystem fungiert als zentrale Steuerungskomponente und koordiniert die Interaktion zwischen den einzelnen Services.

Die Datenhaltung erfolgt über eine relationale Datenbank auf Basis von PostgreSQL. Der Systemzustand wird soweit möglich vollständig in der Datenbank persistiert. Dadurch wird sichergestellt, dass das System auch bei einem Neustart konsistent bleibt und keine Zustandsinformationen verloren gehen.

Die Kommunikation mit der Datenbank erfolgt ausschliesslich über die Anwendungsschicht. Direkte Zugriffe aus der Präsentationsschicht sind nicht vorgesehen. Die Datenbank enthält alle relevanten Entitäten wie Tickets, Parkplätze, Benutzer und Zahlungen.

Externe Systeme wie das Zahlungssystem und das Buchhaltungssystem werden im Rahmen dieses Projekts nicht real angebunden, sondern durch Mock-Klassen simuliert.

Der Datenfluss im System beginnt mit einer Benutzeraktion im Web-Frontend. Die Anfrage wird an die Anwendungsschicht übergeben, dort verarbeitet und bei Bedarf durch Datenbankzugriffe oder Aufrufe von Services ergänzt. Anschliessend wird das Ergebnis an die Präsentationsschicht zurückgegeben und dem Benutzer angezeigt.

Die Architektur ist bewusst einfach gehalten und verzichtet auf komplexe Verteilung oder Microservices. Stattdessen wird eine monolithische Struktur verwendet, die alle Komponenten in einer Anwendung vereint. Diese Entscheidung reduziert die Komplexität und ist für den Umfang des Projekts ausreichend.

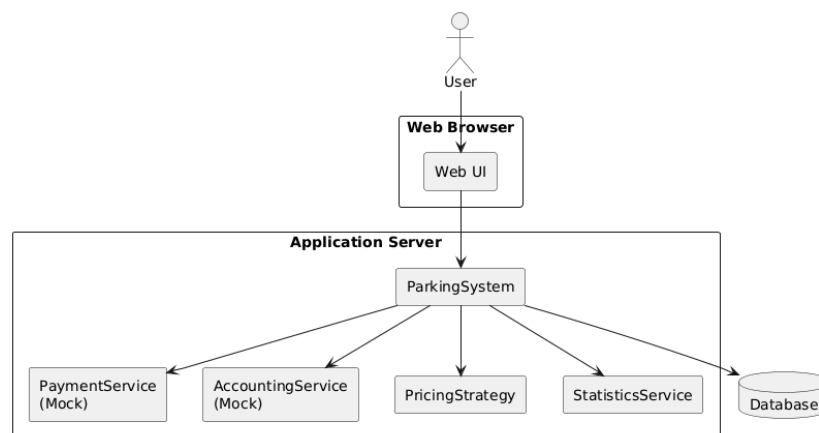


Abbildung 13: Architekturdiagramm

3.10. Testkonzept

Das Testkonzept beschreibt das Vorgehen zur Überprüfung der funktionalen und nicht-funktionalen Anforderungen des Parkhaus-Systems. Ziel ist es, die korrekte Umsetzung der definierten Anforderungen sicherzustellen und Fehler frühzeitig zu erkennen.

3.10.1. Vorgehen

Die Tests werden schrittweise während der Entwicklung durchgeführt. Zunächst werden einzelne Komponenten isoliert getestet, anschliessend erfolgt die Überprüfung des Zusammenspiels mehrerer Komponenten. Abschliessend werden zentrale Anwendungsfälle aus Sicht des Benutzers getestet.

Der Fokus liegt auf der Validierung der Geschäftslogik in der Anwendungsschicht sowie auf der korrekten Verarbeitung und Speicherung von Daten in der Datenbank. Externe Systeme werden über Mock-Klassen simuliert, wodurch deren Verhalten gezielt gesteuert werden kann.

Die Tests orientieren sich direkt an den definierten Anforderungen. Jede funktionale Anforderung wird mindestens durch einen Testfall abgedeckt. Nicht-funktionale Anforderungen werden durch gezielte Szenarien überprüft, beispielsweise durch wiederholte Ausführung oder Messung von Reaktionszeiten.

3.10.2. Testobjekte

Die folgenden Komponenten werden getestet:

- Zentrale Steuerung (ParkingSystem)
- Ticket-Logik (Erstellung, Status, Ausfahrt)
- Parkplatzlogik (Zuweisung, Freigabe)
- Gebührenberechnung (PricingStrategy)
- Zahlungsabwicklung (PaymentService, Mock)
- Persistenz (Datenbankzugriffe und Konsistenz)
- Statistikfunktionen (StatisticsService)

3.10.3. Testfälle

ID	Beschreibung	Erwartetes Ergebnis	Referenz
TC-01	Einfahrt erzeugt Ticket	Ticket wird erstellt und gespeichert	F-04, F-05
TC-02	Parkplatz wird zugewiesen	Freier Parkplatz wird belegt	F-07, F-16
TC-03	Parkdauer wird berechnet	Zeitdifferenz ist korrekt	F-09
TC-04	Gebühr wird korrekt berechnet	Betrag entspricht Tarif	F-10, F-11
TC-05	Viertelstundenabrechnung	Rundung erfolgt korrekt	F-12
TC-06	Tagespauschale greift	Pauschale wird angewendet	F-13
TC-07	Dauermieter Zugang erlaubt	Schranke öffnet	F-06
TC-08	Dauermieter gesperrt	Zugang verweigert	F-14
TC-09	Bezahlung erfolgreich	Ticket wird als bezahlt markiert	F-10
TC-10	Bezahlung fehlgeschlagen	Fehlermeldung wird angezeigt	NF-01
TC-11	Ausfahrt mit gültigem Ticket	Ausfahrt wird erlaubt	F-15
TC-12	Ausfahrt ohne Zahlung	Ausfahrt wird verweigert	F-10
TC-13	Statistik wird erzeugt	Daten werden korrekt angezeigt	F-17
TC-14	Monatsumsatz korrekt	Umsatz stimmt	F-18
TC-15	System reagiert schnell	Antwortzeit < definierter Schwelle	NF-02
TC-16	System bleibt stabil	Keine Abstürze bei Wiederholung	NF-01
TC-17	Daten bleiben konsistent	Keine inkonsistenten Zustände	NF-04, NF-07
TC-18	Benutzeroberfläche verständlich	Bedienung ohne Anleitung möglich	NF-03
TC-19	Spezifikationskonforme Wochentagstarife	Tageszeitslot wird korrekt angewendet	F-11
TC-20	Tarifwechsel an Slotgrenze (Cross-Slot-Abrechnung)	Tarif zu Beginn jeder Viertelstunde gilt für diese Viertelstunde	F-11, F-12
TC-21	Grenzwert 24 Stunden: Studententarif statt Tagespauschale	Tagespauschale greift erst ab mehr als 24 Stunden	F-13
TC-22	Feiertagstarife überschreiben Wochentagstarife	Feiertagstarif wird anstelle des Wochentagstarifs angewendet	F-11

Tabelle 23: Testfälle

3.11. Einführungskonzept

Die Einführung des Systems erfolgt im Rahmen der Projektabgabe und beschränkt sich auf die Bereitstellung der erarbeiteten Ergebnisse. Eine produktive Inbetriebnahme ist nicht vorgesehen.

Im Zentrum der Einführung steht die Dokumentation des Systems. Diese beschreibt die Anforderungen, die Architektur, die Umsetzung sowie die wichtigsten Abläufe. Sie dient dazu, das System nachvollziehbar darzustellen und einen Überblick über die getroffenen Entscheidungen zu geben.

Zusätzlich wird eine abschliessende Präsentation durchgeführt. In dieser werden die wichtigsten Aspekte des Projekts vorgestellt. Dazu gehören insbesondere die Zielsetzung, die gewählte Lösung, zentrale Funktionen sowie ausgewählte technische Entscheidungen. Die Präsentation dient dazu, das Verständnis für das System zu vermitteln und die erarbeiteten Inhalte kompakt zusammenzufassen.

Eine Schulung von Benutzern oder ein Rollout in eine reale Umgebung findet nicht statt. Das System wird ausschliesslich im Rahmen der Projektarbeit demonstriert.

3.12. Migrationskonzept

Ein Migrationskonzept ist für dieses Projekt nicht erforderlich, da kein tatsächlich bestehendes System abgelöst wird und keine Altdaten übernommen werden müssen.

Das Parkhaus-System wird als eigenständige Lösung entwickelt und startet ohne vorhandene Datenbasis. Alle benötigten Daten werden entweder während der Nutzung erzeugt oder können zu Testzwecken initial angelegt werden.

Aufgrund dieses Szenarios sind keine Massnahmen zur Datenübernahme, Datenbereinigung oder Systemumstellung notwendig.

3.13. GUI-Design

Das GUI des Parkhaus-Systems wird als Web-Anwendung umgesetzt und über einen Browser bedient. Ziel ist eine einfache und klare Benutzeroberfläche, die die wichtigsten Funktionen ohne lange Einarbeitung zugänglich macht.

Die Benutzeroberfläche ist in logisch getrennte Bereiche aufgebaut, die sich an den zentralen Anwendungsfällen orientieren. Dazu gehören insbesondere Einfahrt, Ausfahrt, Bezahlung, Verwaltung sowie Auswertungen. Die Navigation erfolgt über eine klare Struktur, sodass die einzelnen Funktionen schnell erreichbar sind.

Die Interaktion erfolgt über einfache Eingabemasken und übersichtliche Anzeigen. Benutzer sollen ohne zusätzliche Erklärung erkennen können, welche Aktionen möglich sind. Wichtige Informationen wie Parkstatus, Gebühren oder Fehlermeldungen werden direkt sichtbar dargestellt. Die Oberfläche vermeidet unnötige Komplexität und konzentriert sich auf die wesentlichen Abläufe.

Für die Darstellung von Parkplätzen wird eine visuelle Übersicht verwendet, in der freie und belegte Plätze klar unterscheidbar sind. Dadurch kann der aktuelle Zustand des Parkhauses schnell erfasst werden. Die Anzeige wird regelmässig aktualisiert, sodass Änderungen zeitnah sichtbar sind.

Die Bezahlungsfunktion wird so gestaltet, dass der Ablauf möglichst einfach ist. Nach dem Scannen oder Eingeben eines Tickets werden die berechneten Gebühren angezeigt und die Zahlung kann direkt ausgelöst werden. Rückmeldungen über den Erfolg oder Fehler einer Zahlung werden unmittelbar angezeigt.

Die Verwaltung von Parkhäusern und Dauermietern erfolgt über separate Bereiche. Dort können Konfigurationen vorgenommen und Daten angepasst werden. Die Eingaben werden validiert, um fehlerhafte Daten zu vermeiden.

Nicht-funktionale Anforderungen wie Benutzbarkeit und Performance werden im GUI berücksichtigt, indem die Oberfläche schnell reagiert und klar strukturiert ist. Aktionen sollen ohne merkliche Verzögerung ausgeführt werden.

4. Realisierung

4.1. Programmierumgebung / Programmierrichtlinien

4.1.1. Programmierumgebung

Die Umsetzung des Prototyps erfolgt als Webanwendung unter Verwendung der Programmiersprache Elixir und des Webframeworks Phoenix in Kombination mit LiveView. Die Wahl dieser Technologien orientiert sich an etablierten Konventionen im Elixir-Ökosystem und unterstützt insbesondere die Entwicklung interaktiver, zustandsbehafteter Weboberflächen ohne komplexe clientseitige Logik.

Als Laufzeitumgebung dient die Erlang VM (BEAM). Die BEAM wurde ursprünglich für Telekommunikationssysteme entwickelt, bei denen hohe Verfügbarkeit und gleichzeitige Verarbeitung vieler unabhängiger Vorgänge zentrale Anforderungen sind. Sie ermöglicht die parallele Ausführung von Millionen leichtgewichtiger Prozesse, die vollständig voneinander isoliert sind. Tritt in einem Prozess ein Fehler auf, bleibt der Rest des Systems davon unberührt. Für dieses Projekt bedeutet das konkret: Mehrere Parkvorgänge können gleichzeitig abgewickelt werden, ohne dass sich die Prozesse gegenseitig beeinflussen. Ein Fehler bei einer Einfahrt oder Zahlung gefährdet nicht die Stabilität des gesamten Systems.

Für die Persistenz wird eine relationale Datenbank auf Basis von PostgreSQL eingesetzt. Der Datenzugriff erfolgt über Ecto, welches als Standardbibliothek für Datenbankinteraktionen im Phoenix-Umfeld gilt.

Das Projekt wird mit dem Build-Tool Mix verwaltet, welches ebenfalls integraler Bestandteil von Elixir ist. Die Versionsverwaltung erfolgt mit Git.

Die Anwendung wird lokal entwickelt und über einen Webbrowser ausgeführt. Eine produktive Deployment-Umgebung ist nicht Bestandteil dieser Arbeit, da der Fokus auf einem funktionsfähigen Prototyp liegt.

4.1.2. Projektstruktur

Die Struktur des Projekts folgt den durch Phoenix vorgegebenen Konventionen. Diese sehen eine klare Trennung zwischen Webschicht und Geschäftslogik vor, was auch dem erarbeiteten Konzept entspricht.

Die Geschäftslogik wird in sogenannten Kontext-Modulen innerhalb des lib-Verzeichnisses implementiert. Diese Module kapseln zusammengehörige Funktionalitäten und stellen eine definierte Schnittstelle nach aussen bereit. Die Webschicht befindet sich im Verzeichnis lib/parking_web und umfasst LiveViews, Controller sowie zugehörige Templates.

Datenbankschemata und Migrationen werden im Verzeichnis priv/repo abgelegt. Eine Migration ist ein versioniertes Skript, das eine Änderung am Datenbankschema beschreibt, z.B. das Erstellen einer Tabelle oder das Hinzufügen einer Spalte. Migrationen werden sequenziell ausgeführt und erlauben es, den Datenbankzustand jederzeit aus dem Quellcode herzustellen oder schrittweise weiterzuentwickeln. Gegenüber einer direkten Anpassung der Datenbank hat dieser Ansatz den Vorteil, dass Schemaänderungen versioniert und nachvollziehbar im Quellcode festgehalten sind. Jede Änderung kann so auf einer neuen Umgebung reproduziert werden, ohne dass die Datenbankstruktur manuell nachgezogen werden muss.

Tests befinden sich im Verzeichnis test und orientieren sich strukturell an den implementierten Modulen.

Diese Struktur entspricht den offiziellen Phoenix-Konventionen und unterstützt eine klare Organisation sowie eine gute Wartbarkeit des Codes.

4.1.3. Programmierrichtlinien

Die Implementierung orientiert sich an den Richtlinien von Elixir. Funktionen werden klein gehalten und erfüllen jeweils eine klar abgegrenzte Aufgabe. Die Kapselung von Logik erfolgt über Module, welche über wohldefinierte Schnittstellen miteinander interagieren.

Die Benennung von Modulen erfolgt in PascalCase, während Funktionen und Variablen in snake_case geschrieben werden. Diese Namenskonventionen entsprechen den üblichen Vorgaben der Elixir-Community.

Fehler werden nicht über Exceptions gesteuert, sondern über explizite Rückgabewerte in Form von Tupeln. Üblich ist die Verwendung von Konstrukten wie `{:ok, result}` und `{:error, reason}`. Dadurch bleibt der Kontrollfluss nachvollziehbar und funktional.

Validierungen von Daten erfolgen primär über Ecto Changesets. Diese ermöglichen eine zentrale Definition von Regeln und stellen sicher, dass nur konsistente Daten persistiert werden.

Der Zugriff auf die Datenbank erfolgt ausschliesslich über das Repository-Modul von Ecto. Direkte SQL-Abfragen werden vermieden, um die Abstraktionsebene beizubehalten und die Wartbarkeit zu erhöhen.

Die Trennung von Verantwortlichkeiten wird konsequent umgesetzt. Die Webschicht ist ausschliesslich für die Interaktion mit dem Benutzer zuständig, während die Geschäftslogik vollständig in den Kontext-Modulen implementiert ist. Dadurch bleibt die Anwendung modular und testbar.

Für die Benutzeroberfläche wird LiveView eingesetzt. Der Zustand der Anwendung wird serverseitig gehalten und über Events aktualisiert. Diese Vorgehensweise entspricht der von Phoenix vorgesehenen Architektur und reduziert die Komplexität im Frontend.

Externe Systeme wie Zahlungs- oder Buchhaltungsdienste werden über klar definierte Schnittstellen abstrahiert. Im Rahmen dieses Prototyps werden diese Systeme nicht real angebunden, sondern durch entsprechende Module simuliert.

4.1.4. Testbarkeit und Qualität

Die Teststrategie orientiert sich an den Möglichkeiten des Elixir-Testframeworks ExUnit. Zentrale Teile der Geschäftslogik werden durch automatisierte Tests überprüft. Dabei liegt der Fokus insbesondere auf kritischen Funktionen wie der Gebührenberechnung oder der Parkplatzzuweisung.

Abhängigkeiten zu externen Systemen werden durch Mock-Implementierungen ersetzt, sodass Tests unabhängig und reproduzierbar ausgeführt werden können.

Durch die klare Trennung von Logik und Präsentation sowie die Nutzung von Kontext-Modulen wird eine gute Testbarkeit der Anwendung erreicht.

4.2. Softwareaufbau

4.2.1. Modulstruktur

Die Anwendung ist gemäss den Phoenix-Konventionen in zwei klar getrennte Bereiche gegliedert: die Geschäftslogik unter `lib/parking/` und die Webschicht unter `lib/parking_web/`. Innerhalb dieser Bereiche sind die Verantwortlichkeiten auf spezialisierte Module aufgeteilt.

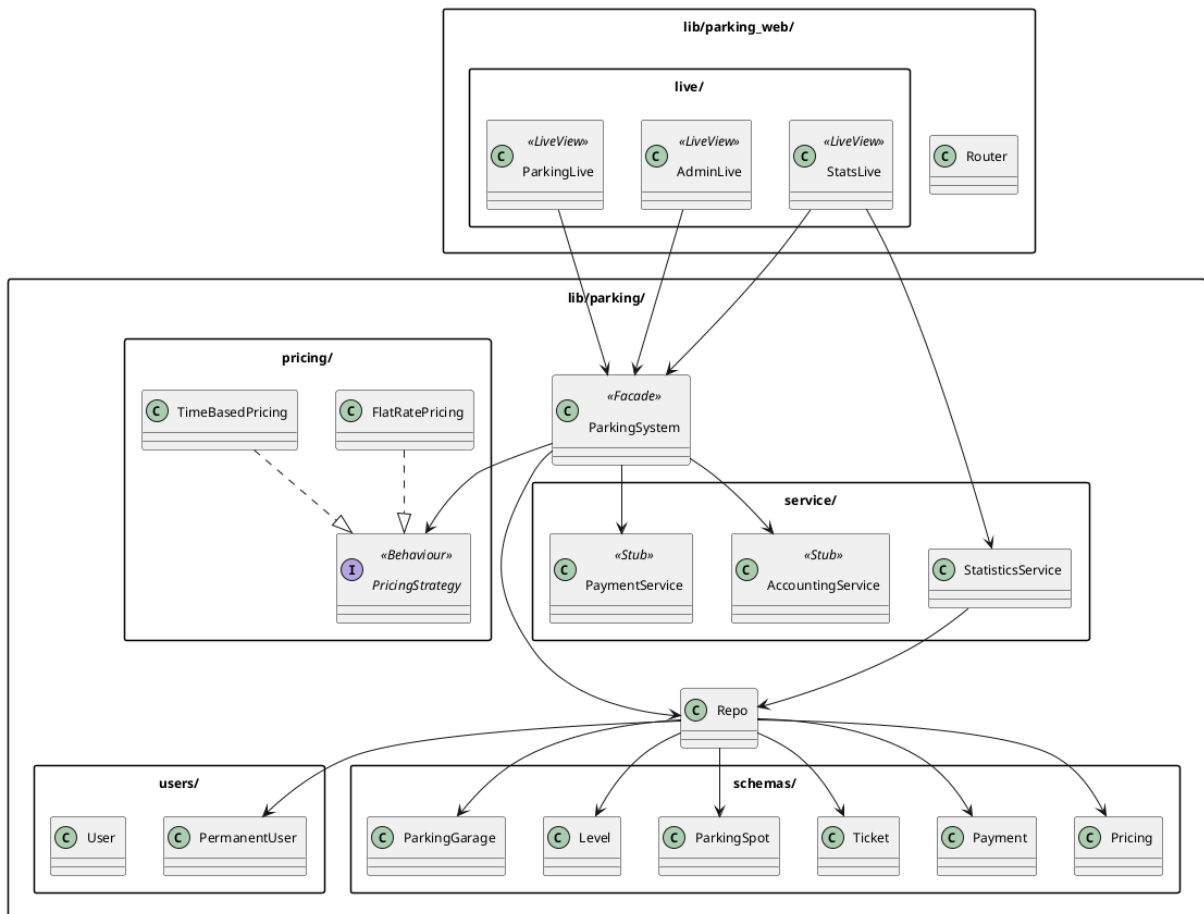


Abbildung 14: Modulübersicht

4.2.2. Geschäftslogik

Das Modul `Parking.ParkingSystem` bildet den zentralen Einstiegspunkt für alle fachlichen Operationen. Es koordiniert die übrigen Module und stellt eine definierte Schnittstelle für die Webschicht bereit. Dazu gehören das Erstellen und Verwalten von Tickets, die Authentifizierung und Abwicklung von Dauermietern, die Gebührenberechnung sowie die Freigabe von Parkplätzen bei der Ausfahrt. Das Modul entspricht dem Fassade-Muster: Die Webschicht interagiert ausschliesslich mit diesem Modul und nicht direkt mit den darunter liegenden Schemata oder Diensten.

Die Domänenentitäten sind als Ecto-Schemata implementiert.

Parking.ParkingGarage

Repräsentiert ein Parkhaus und enthält den Namen sowie Verknüpfungen zu Stockwerken und Tarifen.

Parking.Level

Stellt ein Stockwerk innerhalb eines Parkhauses mit Stockwerksnummer und Parkplatzliste dar.

Parking.ParkingSpot

Entspricht einem einzelnen Parkplatz und hält dessen Belegungsstatus sowie eine lesbare Platznummer (`number`). Die Platznummer wird nach dem Schema $\text{Stockwerksnummer} \times 100 + \text{laufende Nummer}$ vergeben, sodass Platz 3 auf Stockwerk 2 die Nummer 203 trägt. Im Konzept war ein dreistufiger Lebenszyklus (frei, reserviert, belegt) vorgesehen. In der Umsetzung wurde dieser auf den boolean Wert Belegungsstatus (`is_occupied`) vereinfacht, da eine explizite Reservierungsphase im Prototyp nicht benötigt wird. Die Zuweisung und Belegung eines Parkplatzes erfolgt beim Erstellen des Tickets.

Parking.Ticket

Ist das Parkticket mit Einfahrtszeit, Ausfahrtszeit, Bezahlstatus sowie Verknüpfungen zu Parkplatz, Tarif und optional einem Dauermieter.

Parking.Payment

Speichert Zahlungsdatensätze mit Betrag und Zeitstempel.

Parking.Pricing

Enthält den Tarif-Typ eines Parkhauses und verknüpft die zugehörige Konfiguration über typisierte Untertabellen. Unterstützte Typen sind `time_based`, `daily_rate` und `monthly_rent`. Die Konfigurationsdetails werden in separaten Tabellen (`time_based_pricing`, `daily_rate_pricing`, `monthly_rent_pricing`) gespeichert, die jeweils nur die für den jeweiligen Typ relevanten Felder enthalten. Die Monatsmiete ist damit pro Parkhaus konfigurierbar.

Parking.Users.PermanentUser

Repräsentiert den Dauermieter mit Zugangscode, Sperrstatus, Mietdaten und einem fixen Parkplatz.

Die Entität `Parking.Users.OccasionalUser` existiert als Ecto-Schema zur Vervollständigung des Datenmodells, enthält jedoch ausser der ID-Verknüpfung zur `User`-Tabelle keine weiteren Felder. Bei der Einfahrt eines Gelegenheitsnutzers wird kein `OccasionalUser`-Datensatz angelegt — der Parkvorgang wird vollständig über das Ticket abgebildet.

4.2.3. Preisberechnungsstrategie

Die Gebührenberechnung ist über ein Elixir-Behaviour (`Parking.Pricing.PricingStrategy`) abstrahiert. Dieses definiert die Schnittstelle `calculate/2`, welche eine Strategie-Struktur und ein Ticket entgegennimmt und den geschuldeten Betrag zurückgibt.

`Parking.Pricing.TimeBasedPricing` berechnet die Gebühr anhand der Parkdauer und konfigurierbarer Zeitslots. Die Abrechnung erfolgt auf Viertelstundenbasis, wobei der zu Beginn der jeweiligen Viertelstunde geltende Tarif für die gesamte Viertelstunde gilt. Für Wochenenden und Feiertage können separate Zeitslot-Listen konfiguriert werden. Feiertage werden als Datumsliste in der Tarif-Konfiguration hinterlegt. Überschreitet die Parkdauer 24 Stunden, wird automatisch auf die Tagespauschale umgestellt. `Parking.Pricing.FlatRatePricing` bietet eine vereinfachte Alternative, die eine Tagespauschale unabhängig von der Tageszeit anwendet. Die Konfiguration der jeweiligen Strategie wird aus den typisierten Untertabellen gelesen.

Die Wahl der anzuwendenden Strategie erfolgt im `ParkingSystem` anhand des `type`-Felds des Tarif-Datensatzes.

4.2.4. Serviceschicht

Externe Systeme werden über dedizierte Servicemodule angebunden.

Parking.Services.PaymentService

Simuliert die Anbindung an ein externes Zahlungssystem. Im Prototyp gibt der Dienst stets eine Erfolgsantwort zurück. Für Tests existiert ein `FailingStub`, der eine fehlgeschlagene Zahlung simuliert.

Parking.Services.AccountingService

Simuliert die Buchung von Transaktionen in einem Buchhaltungssystem und ist im Prototyp als Stub implementiert.

Parking.Services.StatisticsService

Berechnet Umsatzdaten aus den gespeicherten Zahlungsdatensätzen und unterstützt Berechnungen nach Zeitraum, Monat und Jahr, jeweils aufgeschlüsselt nach Kundenkategorie.

Das aktiv genutzte Servicemodul wird über die Applikationskonfiguration bestimmt, was den Austausch gegen Test-Stubs ermöglicht, ohne den Produktionscode zu verändern.

4.3. GUI-Implementierung

Die Benutzeroberfläche ist als Phoenix LiveView-Anwendung umgesetzt. Der Zustand der Oberfläche wird vollständig serverseitig verwaltet. Benutzerinteraktionen werden als Events an den Server übermittelt und führen zu einem gezielten Update der Seite. Die Anwendung ist über drei Routen erreichbar: / und /garage/:garage_id führen zur Parkhaus-Ansicht, /admin zur Administrationsoberfläche und /stats zur Statistik-Ansicht.

4.3.1. Parkhaus-Ansicht (ParkingLive)

Die Parkhaus-Ansicht bildet die primäre Schnittstelle für den Parkierungsprozess. Beim Laden der Seite wird das erste verfügbare Parkhaus ausgewählt. Über ein Auswahlménü kann zwischen mehreren Parkhäusern gewechselt werden.

Für Gelegenheitsnutzer simuliert ein Klick auf die Einfahrtsschaltfläche das Drücken des Knopfs an der Eingangsschranke. Das System erstellt ein Ticket, weist einen freien Parkplatz zu und zeigt die Ticketdetails (UUID, Einfahrtszeit, zugewiesener Parkplatz) sowie die bis dahin aufgelaufene Gebühr direkt an. Die angezeigte Gebühr wird serverseitig bei jeder Interaktion neu berechnet. Nach der Bezahlung gibt die Ausfahrt-Schaltfläche den Parkplatz frei und markiert die Ausfahrtszeit. Über eine Scan-Funktion kann ein bestehendes Ticket anhand der UUID geladen werden, um dessen Status einzusehen oder die Bezahlung und Ausfahrt nachträglich vorzunehmen.

Für Dauermieter authentifiziert sich der Nutzer mit seinem Zugangscode. Das System prüft den Code sowie den Zahlungsstatus. Die Sperrlogik unterscheidet zwei Fälle: Ist die Miete zwei oder mehr Monate im Rückstand, erfolgt die Sperrung sofort. Ist die Miete genau einen Monat im Rückstand, gilt eine Frist bis zum 15. des laufenden Monats. Nach erfolgreicher Anmeldung werden Einfahrt und Ausfahrt separat ausgelöst, wobei ein aktives Ticket erstellt beziehungsweise abgeschlossen und der Parkplatz entsprechend belegt oder freigegeben wird.

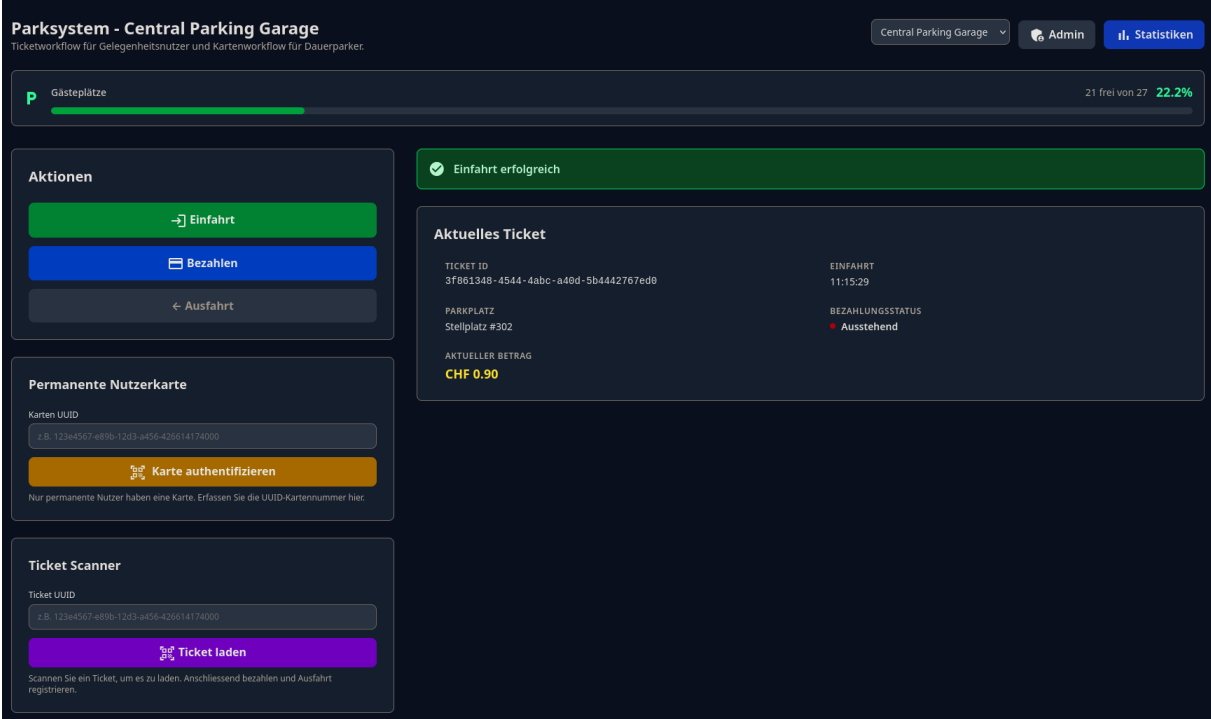


Abbildung 15: Einfahrt Gelegenheitsnutzer

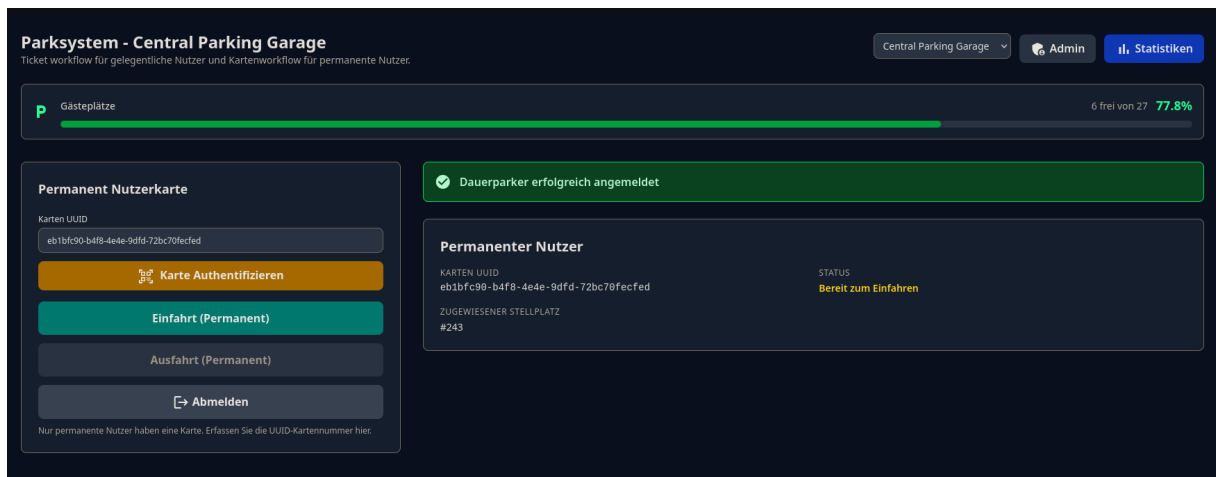


Abbildung 16: Dauermieter-Bereich

4.3.2. Administrationsoberfläche (AdminLive)

Die Administrationsoberfläche ist durch ein Passwort geschützt. Erst nach erfolgreicher Anmeldung werden die Verwaltungsfunktionen freigeschaltet und die Dauermieterliste des ausgewählten Parkhauses angezeigt. Die Tabelle enthält Name, Zugangscode, zugewiesenen Parkplatz, Sperrstatus sowie das Datum, bis zu dem die Miete bezahlt ist.

Ein neuer Dauermieter wird mit automatisch generiertem UUID-Zugangscode angelegt. Das System weist ihm automatisch den nächsten freien Parkplatz zu und zeigt den generierten Code nach der Erstellung an. Bei der Erstellung wird die erste Monatsmiete sofort als Ticket- und Zahlungsdatensatz erfasst, sodass der Dauermieter ab dem ersten Tag Zugang erhält. Die Miete eines Dauermieters kann als bezahlt markiert werden, wodurch das `rent_paid_until`-Datum um einen Monat verlängert, ein Ticket sowie ein Zahlungsdatensatz für die Miete erstellt und ein allfälliger Sperrstatus aufgehoben wird. Zusätzlich kann der Sperrstatus einzelner Dauermieter manuell umgeschaltet werden. Einem Dauermieter kann nachträglich ein anderer freier Parkplatz zugewiesen werden, sofern er aktuell nicht parkiert.

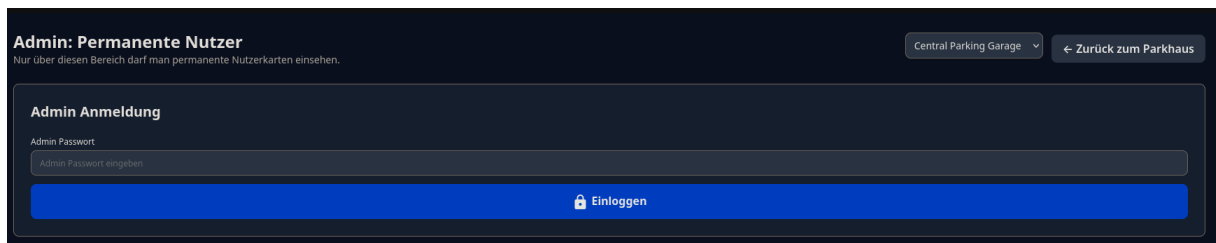


Abbildung 17: Admin Login

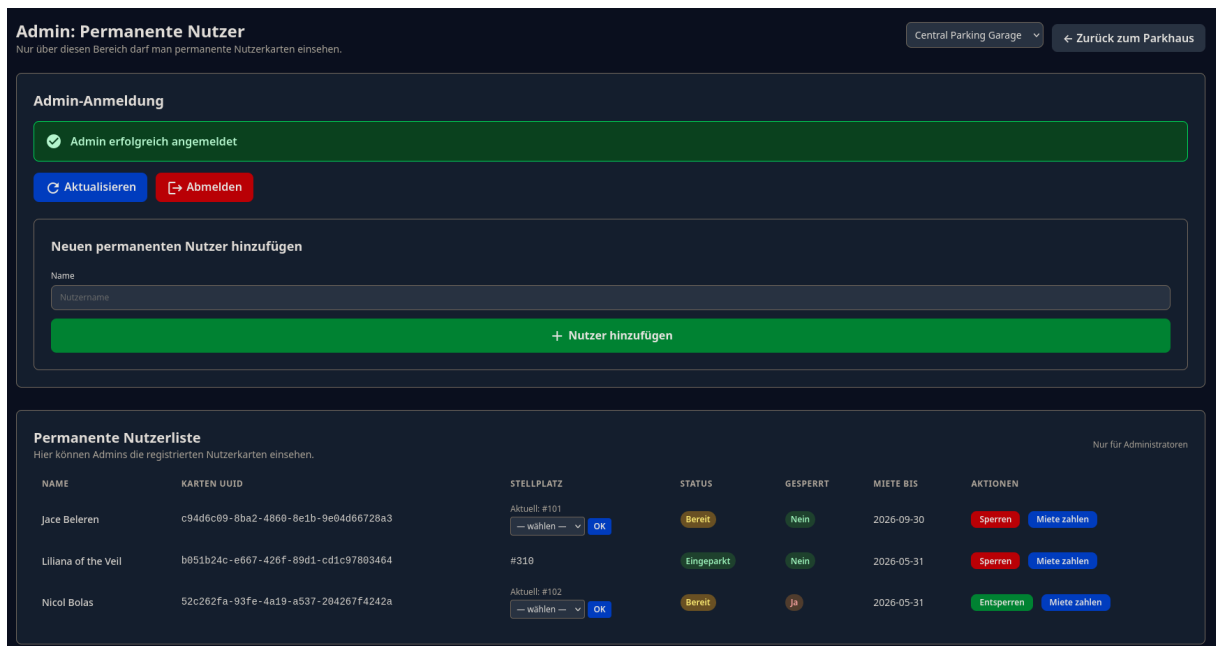


Abbildung 18: Admin Ansicht

4.3.3. Statistik-Ansicht (StatsLive)

Die Statistik-Ansicht zeigt Belegungs- und Umsatzdaten für das ausgewählte Parkhaus. Die Daten werden beim Laden der Seite berechnet und sind per Parkhaus-Auswahlmenü filterbar.

Die Belegungsstatistik umfasst die Gesamtanzahl Parkplätze, aufgeteilt nach Dauermieter-Plätzen, belegten Gast-Plätzen und freien Gast-Plätzen, sowie die Auslastungsquote in Prozent. Der Monatsumsatz zeigt den Umsatz des laufenden Monats, der Jahresumsatz jenen des laufenden Jahres. Beide werden nach Gelegenheitsnutzern und Dauermietern aufgeschlüsselt dargestellt. Die für das Parkhaus erfassten Tarife, Tagespauschale und Dauermieten werden aufgelistet.

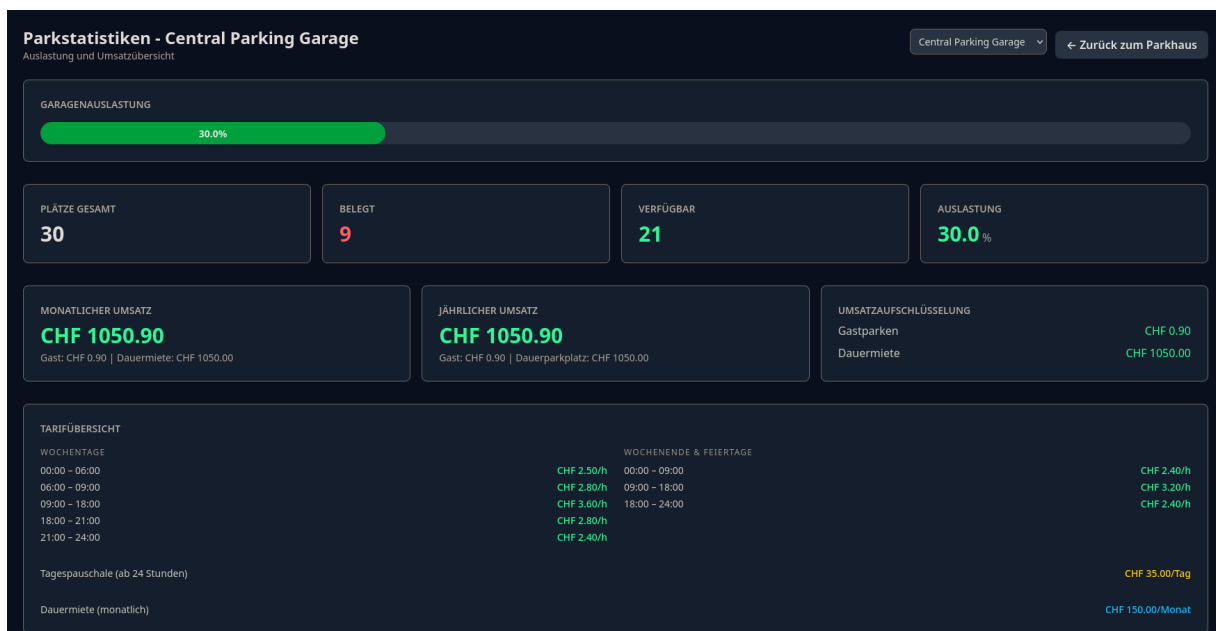


Abbildung 19: Statistik-Ansicht

Zusätzlich zeigt die Seite eine tabellarische Übersicht aller Stockwerke und Parkplätze, wobei jeder Platz mit seinem Typ (Gast frei, Gast belegt, Dauermieter frei, Dauermieter belegt) gekennzeichnet ist.

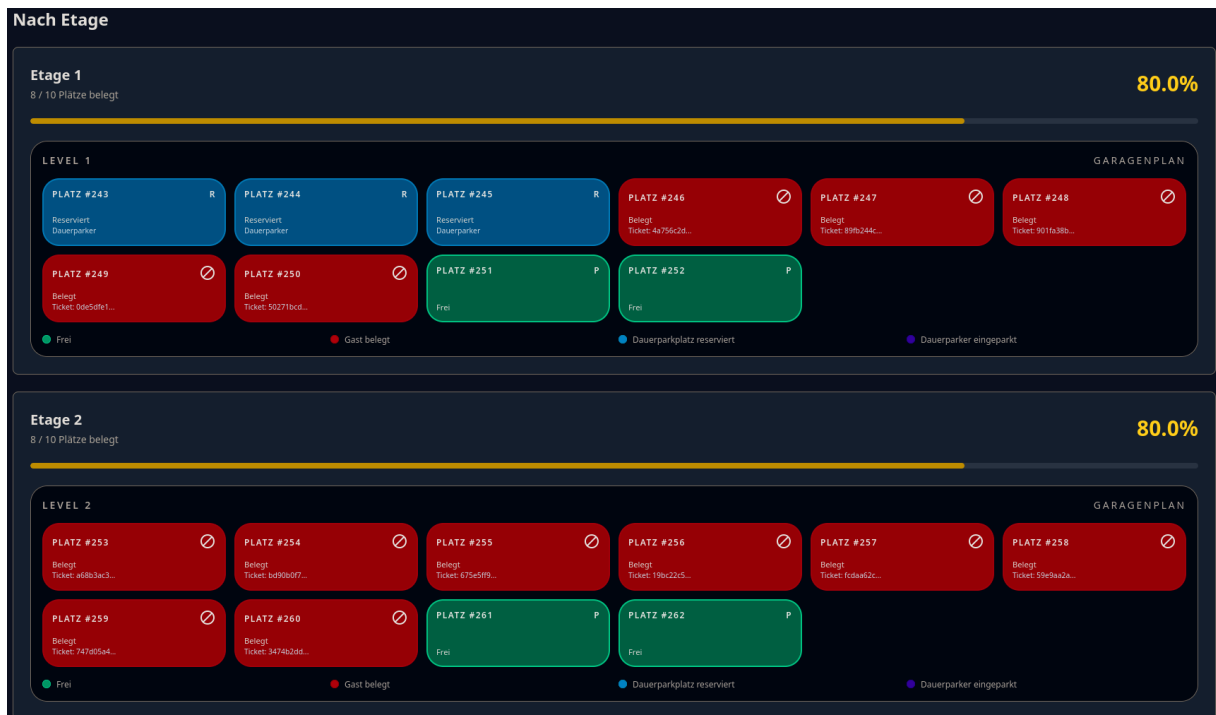


Abbildung 20: Etagen Ansicht

4.4. Datenbankimplementierung und -anbindung

4.4.1. Datenbankschema

Die Persistenz erfolgt über eine relationale PostgreSQL-Datenbank. Das Schema wird durch Ecto-Migrationen verwaltet, welche eine nachvollziehbare und reproduzierbare Datenbankstruktur sicherstellen.

Tabelle	Beschreibung
parking_garage	Parkhaus mit Name
level	Stockwerk mit Nummer und Verweis auf das Parkhaus
parking_spot	Parkplatz mit lesbarer Platznummer, Belegungsstatus und Verweis auf das Stockwerk
occasional_user	Gelegenheitsnutzer als Stub-Entität; enthält nur die ID-Verknüpfung zur User-Tabelle, wird im Parkvorgang nicht aktiv befüllt
ticket	Parkticket mit UUID-Primärschlüssel, Ein-/Ausfahrtszeit, Bezahlstatus sowie Verweisen auf Parkplatz, Tarif und optional Dauermieter
pricing	Tarif-Eintrag mit Typ (time_based, daily_rate, monthly_rent), verknüpft mit einem Parkhaus
time_based_pricing	Konfiguration für zeitbasierte Tarife: Zeitslots, Wochenend- und Feiertagstarife, Tagespauschale
daily_rate_pricing	Konfiguration für Tagespauschalen-Tarife: einheitliche Tagesrate
monthly_rent_pricing	Konfiguration für Monatsmiete pro Parkhaus: monatlicher Mietbetrag
payment	Zahlungsdatensatz mit Betrag, Zeitstempel und Verweis auf das Ticket
user	Basisentität für alle Benutzer mit Typenfeld
permanent_user	Dauermieter mit Zugangscode, Sperrstatus, Mietdaten und zugewiesenem Parkplatz
settings	Schlüssel-Wert-Tabelle für systemweite Konfigurationsparameter wie das Admin-Passwort

Tabelle 24: Datenbankstruktur

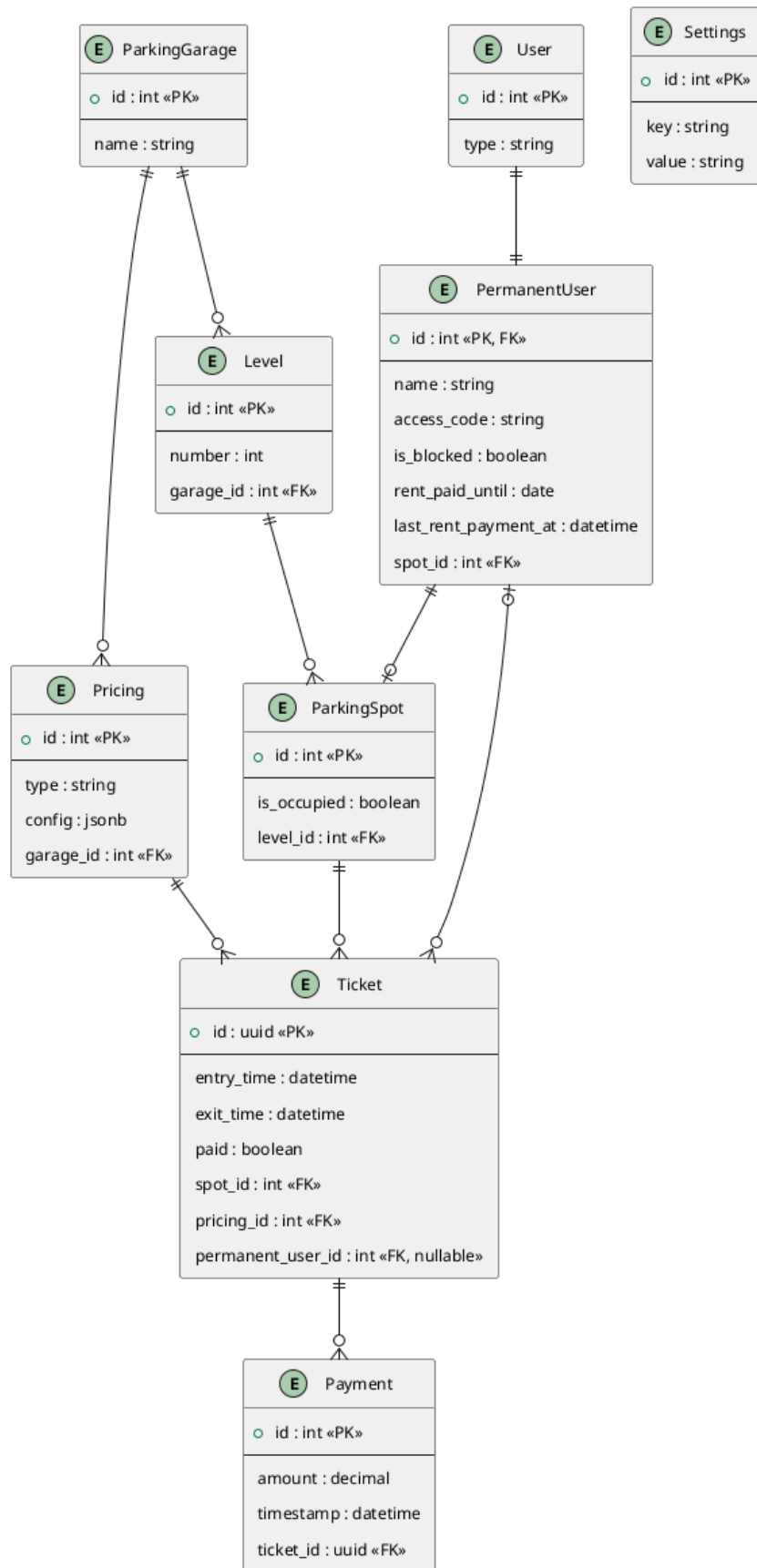


Abbildung 21: Entity-Relationship-Diagramm

4.4.2. Designentscheidungen

4.4.2.1. UUID als Primärschlüssel für Tickets

Tickets verwenden einen UUID-Primärschlüssel (`binary_id`). Dies ermöglicht die sichere Übergabe der Ticket-ID an den Benutzer (z.B. zur Anzeige am Bildschirm oder als QR-Code), ohne dass sequentielle IDs erraten werden können.

4.4.2.2. Typisierte Untertabellen für Tarife

Jeder Tarif-Typ (`time_based`, `daily_rate`, `monthly_rent`) besitzt eine eigene Untertabelle mit den jeweils relevanten Feldern. Dies ermöglicht Datenbankvalidierungen auf Feldebene und klarere Abfragen. Die Monatsmiete ist als eigener Tarif-Typ pro Parkhaus hinterlegt, sodass verschiedene Parkhäuser unterschiedliche Mietbeträge verwenden können.

4.4.2.3. Migrationsbasierte Schemaevolution

Alle Schemaänderungen werden als Ecto-Migrationen eingecheckt. Dies stellt sicher, dass der Datenbankzustand zu jedem Zeitpunkt aus dem Quellcode reproduziert werden kann.

4.4.3. Datenzugriff

Der Datenzugriff erfolgt ausschliesslich über das Ecto-Repository-Modul (`Parking.Repo`). Direkte SQL-Abfragen werden nicht verwendet. Komplexere Abfragen - etwa die Suche nach dem nächsten freien Parkplatz mit ausgeglichener Verteilung oder die Umsatzauswertung nach Kundenkategorie - werden über die Ecto-Query-DSL formuliert.

Ecto-Changesets werden für alle schreibenden Operationen eingesetzt. Sie zentralisieren die Validierungsregeln und stellen sicher, dass nur konsistente Daten in die Datenbank gelangen.

4.4.4. Externe Systemanbindung

Zahlungssystem und Buchhaltung sind gemäss Aufgabenstellung nicht Bestandteil der Software und werden über Schnittstellen angebunden. Im Prototyp sind diese Systeme als Stub-Module implementiert, welche die Schnittstelle erfüllen, aber keine reale Anbindung besitzen. Für Testzwecke kann der `PaymentService` über die Applikationskonfiguration durch einen `FailingStub` ersetzt werden, der eine fehlgeschlagene Zahlung zurückgibt.

4.5. Testprotokoll

4.5.1. Teststrategie

Die Tests werden mit `ExUnit`, dem in Elixir integrierten Testframework, durchgeführt. Für Tests, die Datenbankzugriffe erfordern, wird `Parking.DataCase` verwendet. Dieses setzt jede Testgruppe in eine Datenbanktransaktion, welche nach dem Test zurückgerollt wird. Dadurch sind alle Tests voneinander isoliert und hinterlassen keinen Zustand in der Testdatenbank. Externe Abhängigkeiten werden durch Stub-Implementierungen ersetzt, sodass Tests ohne reale Systemanbindung ausgeführt werden können.

4.5.2. Testfälle

ID	TC-01
Beschreibung	Einfahrt erzeugt Ticket
Vorgehen	<code>ParkingSystem.create_ticket/2</code> wird mit einer gültigen Garage-ID und Tarif-ID aufgerufen.
Erwartetes Ergebnis	Ticket wird erstellt und gespeichert
Tatsächliches Ergebnis	Ticket mit gültiger UUID und Einfahrtszeit wird erstellt. Der zugehörige Parkplatz wird als belegt markiert.
Status	Bestanden
Referenz	F-04, F-05

Tabelle 25: Testergebnis TC-01

ID	TC-02
Beschreibung	Parkplatz wird zugewiesen
Vorgehen	Nach der Einfahrt wird der dem Ticket zugewiesene Parkplatz aus der Datenbank gelesen.
Erwartetes Ergebnis	Freier Parkplatz wird belegt
Tatsächliches Ergebnis	Parkplatz ist als belegt markiert (<code>is_occupied: true</code>).
Status	Bestanden
Referenz	F-07, F-16

Tabelle 26: Testergebnis TC-02

ID	TC-03
Beschreibung	Parkdauer wird berechnet
Vorgehen	Ein Ticket mit bekannter Einfahrts- und Ausfahrtszeit (2 Stunden) wird an <code>TimeBasedPricing.calculate/2</code> übergeben.
Erwartetes Ergebnis	Zeitdifferenz ist korrekt
Tatsächliches Ergebnis	Der berechnete Betrag entspricht dem erwarteten Wert auf Basis der Parkdauer.
Status	Bestanden
Referenz	F-09

Tabelle 27: Testergebnis TC-03

ID	TC-04
Beschreibung	Gebühr wird korrekt berechnet
Vorgehen	Tickets mit verschiedenen Parkdauern und Tarif-Konfigurationen werden berechnet: Grundtarif, Tageszeit-Slot und Wochenend-Slot.
Erwartetes Ergebnis	Betrag entspricht Tarif
Tatsächliches Ergebnis	Berechnete Beträge stimmen in allen drei Szenarien mit den erwarteten Werten überein.
Status	Bestanden
Referenz	F-10, F-11

Tabelle 28: Testergebnis TC-04

ID	TC-05
Beschreibung	Viertelstundenabrechnung
Vorgehen	Tickets mit 45 Minuten und 5 Minuten Parkdauer werden an <code>TimeBasedPricing.calculate/2</code> übergeben.
Erwartetes Ergebnis	Rundung erfolgt korrekt
Tatsächliches Ergebnis	45 Minuten werden als 3 vollständige Viertelstunden verrechnet. 5 Minuten werden als 1 Viertelstunde verrechnet.
Status	Bestanden
Referenz	F-12

Tabelle 29: Testergebnis TC-05

ID	TC-06
Beschreibung	Tagespauschale greift
Vorgehen	Ein Ticket mit 25 Stunden Parkdauer wird an <code>TimeBasedPricing.calculate/2</code> übergeben. Die Konfiguration verwendet die spezifikationskonforme Tagespauschale von CHF 35.00.
Erwartetes Ergebnis	Pauschale wird angewendet
Tatsächliches Ergebnis	Es werden 2 Tagespauschalen à CHF 35.00 berechnet (CHF 70.00 total). Der Stundentarif wird nicht angewendet.
Status	Bestanden
Referenz	F-13

Tabelle 30: Testergebnis TC-06

ID	TC-07
Beschreibung	Dauermieter Zugang erlaubt
Vorgehen	<code>ParkingSystem.authenticate_permanent_user/1</code> wird mit dem korrekten Zugangscode eines nicht gesperrten Dauermieters aufgerufen.
Erwartetes Ergebnis	Schranke öffnet
Tatsächliches Ergebnis	Authentifizierung erfolgreich. Dauermieter-Datensatz wird zurückgegeben.
Status	Bestanden
Referenz	F-06

Tabelle 31: Testergebnis TC-07

ID	TC-08
Beschreibung	Dauermieter gesperrt
Vorgehen	Der Dauermieter wird als gesperrt markiert. Anschliessend wird <code>authenticate_permanent_user/1</code> aufgerufen.
Erwartetes Ergebnis	Zugang verweigert
Tatsächliches Ergebnis	<code>{:error, {:blocked, perm_user}}</code> wird zurückgegeben.
Status	Bestanden
Referenz	F-14

Tabelle 32: Testergebnis TC-08

ID	TC-09
Beschreibung	Bezahlung erfolgreich
Vorgehen	<code>ParkingSystem.process_payment/1</code> wird mit einem bestehenden, unbezahlten Ticket aufgerufen.
Erwartetes Ergebnis	Ticket wird als bezahlt markiert
Tatsächliches Ergebnis	Ticket erhält <code>paid: true</code> . Ein Zahlungsdatensatz wird in der Datenbank erstellt.
Status	Bestanden
Referenz	F-10

Tabelle 33: Testergebnis TC-09

ID	TC-10
Beschreibung	Bezahlung fehlgeschlagen
Vorgehen	Der PaymentService wird durch einen FailingStub ersetzt. Anschliessend wird process_payment/1 aufgerufen.
Erwartetes Ergebnis	Fehlermeldung wird angezeigt
Tatsächliches Ergebnis	{:error, :payment_failed} wird zurückgegeben. Die Transaktion wird zurückgerollt. Das Ticket bleibt unbezahlt und der Parkplatz belegt.
Status	Bestanden
Referenz	NF-01

Tabelle 34: Testergebnis TC-10

ID	TC-11
Beschreibung	Ausfahrt mit gültigem Ticket
Vorgehen	Nach Einfahrt und Bezahlung wird ParkingSystem.register_exit/1 aufgerufen.
Erwartetes Ergebnis	Ausfahrt wird erlaubt
Tatsächliches Ergebnis	Ausfahrtszeit wird auf dem Ticket gesetzt. Der Parkplatz wird als frei markiert.
Status	Bestanden
Referenz	F-15

Tabelle 35: Testergebnis TC-11

ID	TC-12
Beschreibung	Ausfahrt ohne Zahlung
Vorgehen	Nach der Einfahrt wird register_exit/1 ohne vorherige Bezahlung aufgerufen.
Erwartetes Ergebnis	Ausfahrt wird verweigert
Tatsächliches Ergebnis	{:error, :payment_required} wird zurückgegeben. Der Parkplatz bleibt belegt.
Status	Bestanden
Referenz	F-10

Tabelle 36: Testergebnis TC-12

ID	TC-13
Beschreibung	Statistik wird erzeugt
Vorgehen	Zahlungsdatensätze für Gelegenheitsnutzer und Dauermieter werden in der Testdatenbank angelegt. StatisticsService.calculate_revenue_by_period/3 wird aufgerufen.
Erwartetes Ergebnis	Daten werden korrekt angezeigt
Tatsächliches Ergebnis	Summen nach Kundenkategorie stimmen mit den erwarteten Werten überein. Gelegenheitsnutzer- und Dauermieter-Umsätze werden korrekt getrennt ausgewiesen.
Status	Bestanden
Referenz	F-17

Tabelle 37: Testergebnis TC-13

ID	TC-14
Beschreibung	Monatsumsatz korrekt
Vorgehen	Zahlungsdatensätze für den laufenden Monat werden angelegt. StatisticsService.calculate_monthly_revenue/3 wird aufgerufen.
Erwartetes Ergebnis	Umsatz stimmt
Tatsächliches Ergebnis	Berechneter Monatsumsatz stimmt mit der Summe der angelegten Zahlungen überein.
Status	Bestanden
Referenz	F-18

Tabelle 38: Testergebnis TC-14

ID	TC-15
Beschreibung	System reagiert schnell
Vorgehen	Manueller Test – Einfahrt, Bezahlung und Ausfahrt werden über die Weboberfläche ausgeführt. Die Reaktionszeit wird beobachtet.
Erwartetes Ergebnis	Antwortzeit < definierter Schwelle
Tatsächliches Ergebnis	Alle Aktionen reagieren unmittelbar. Keine spürbaren Verzögerungen festgestellt.
Status	Bestanden
Referenz	NF-02

Tabelle 39: Testergebnis TC-15

ID	TC-16
Beschreibung	System bleibt stabil
Vorgehen	Manueller Test – Mehrere Einfahrten und Ausfahrten werden in Folge durchgeführt. Das System wird auf Abstürze und Fehler beobachtet.
Erwartetes Ergebnis	Keine Abstürze bei Wiederholung
Tatsächliches Ergebnis	System bleibt stabil. Keine Fehler oder unerwarteten Zustände aufgetreten.
Status	Bestanden
Referenz	NF-01

Tabelle 40: Testergebnis TC-16

ID	TC-17
Beschreibung	Daten bleiben konsistent
Vorgehen	Eine fehlgeschlagene Zahlung wird über den <code>FailingStub</code> simuliert. Anschliessend werden Ticket und Parkplatz aus der Datenbank gelesen.
Erwartetes Ergebnis	Keine inkonsistenten Zustände
Tatsächliches Ergebnis	Ticket bleibt unbezahlt. Parkplatz bleibt belegt. Kein Zahlungsdatensatz wurde erstellt.
Status	Bestanden
Referenz	NF-04, NF-07

Tabelle 41: Testergebnis TC-17

ID	TC-18
Beschreibung	Benutzeroberfläche verständlich
Vorgehen	Manueller Test – Die Weboberfläche wird ohne Anleitung bedient. Alle Kernfunktionen werden ausgeführt.
Erwartetes Ergebnis	Bedienung ohne Anleitung möglich
Tatsächliches Ergebnis	Alle Kernfunktionen (Einfahrt, Bezahlung, Ausfahrt, Dauermieter-Verwaltung) sind intuitiv zugänglich.
Status	Bestanden
Referenz	NF-03

Tabelle 42: Testergebnis TC-18

ID	TC-19
Beschreibung	Spezifikationskonforme Wochentagstarife
Vorgehen	Ein Ticket mit 2 Stunden Parkdauer am Donnerstag (10:00-12:00) wird mit der vollständigen Tarifkonfiguration gemäss Lastenheft Anhang 5.1 an <code>TimeBasedPricing.calculate/2</code> übergeben.
Erwartetes Ergebnis	Tageszeitslot wird korrekt angewendet
Tatsächliches Ergebnis	Der Slot 09:00-18:00 (CHF 3.60/Std.) wird erkannt. Es werden 8 Viertelstunden à CHF 0.90 berechnet (CHF 7.20 total).
Status	Bestanden
Referenz	F-11

Tabelle 43: Testergebnis TC-19

ID	TC-20
Beschreibung	Tarifwechsel an Slotgrenze (Cross-Slot-Abrechnung)
Vorgehen	Ein Ticket mit Einfahrt 17:50 und Ausfahrt 18:10 an einem Wochentag wird an <code>TimeBasedPricing.calculate/2</code> übergeben.
Erwartetes Ergebnis	Der Tarif zu Beginn jeder Viertelstunde gilt für diese Viertelstunde
Tatsächliches Ergebnis	Die Viertelstunde ab 17:45 wird mit CHF 3.60/Std. (Tagstarif, CHF 0.90) abgerechnet, die Viertelstunde ab 18:00 mit CHF 2.80/Std. (Abendtarif, CHF 0.70). Total CHF 1.60.
Status	Bestanden
Referenz	F-11, F-12

Tabelle 44: Testergebnis TC-20

ID	TC-21
Beschreibung	Grenzwert 24 Stunden: Stundentarif statt Tagespauschale
Vorgehen	Ein Ticket mit exakt 24 Stunden Parkdauer wird an <code>TimeBasedPricing.calculate/2</code> übergeben.
Erwartetes Ergebnis	Tagespauschale wird bei exakt 24 Stunden nicht ausgelöst
Tatsächliches Ergebnis	Die Bedingung <code>duration > 86400</code> ist <code>false</code> . Es werden 96 Viertelstunden à CHF 0.75 berechnet (CHF 72.00 total). Die Tagespauschale wird erst ab mehr als 24 Stunden angewendet.
Status	Bestanden
Referenz	F-13

Tabelle 45: Testergebnis TC-21

ID	TC-22
Beschreibung	Feiertagstarife überschreiben Wochentagstarife
Vorgehen	Ein Ticket mit 2 Stunden Parkdauer am 01.01.2026 (Neujahr, Donnerstag) wird mit der vollständigen Tarifkonfiguration an <code>TimeBasedPricing.calculate/2</code> übergeben.
Erwartetes Ergebnis	Feiertagstarif wird angewendet statt Wochentagstarif
Tatsächliches Ergebnis	Der Feiertag wird erkannt. Der Slot 09:00-18:00 (CHF 3.20/Std.) aus den Feiertagstarifen wird angewendet. Es werden 8 Viertelstunden à CHF 0.80 berechnet (CHF 6.40 total). Der Wochentagstarif (CHF 3.60/Std.) wird nicht angewendet.
Status	Bestanden
Referenz	F-11

Tabelle 46: Testergebnis TC-22

5. Fazit / Lessons learned

Die Nachvollziehbarkeit für Project Controlling in dieser Arbeit wäre meiner Meinung nach nicht genügend für die Diplomarbeit. In der Diplomarbeit werde ich die geleisteten Aufwände dokumentieren. Ausserdem werde ich häufiger commits erstellen, was ich etwas vernachlässigte, da ich der einzige Entwickler/Tester in diesem Projekt war.

Die Technologiewahl von Elixir/Phoenix hat sich bewährt. Der Funktionale Ansatz machte das automatische Testen sehr einfach. Bis heute bin ich aber sicher noch kein Experte in Elixir und könnte nicht alle Details genau erklären. Z.B. Ecto für die Datenbankbindung habe ich einfach angewandt, ohne genauer zu hinterfragen wie es funktioniert. Ich werde diese Technologien in der Zukunft wieder verwenden.

Typst + plantuml für die Dokumentation war sehr angenehm. Dank Typst hatte ich nie Probleme bei der Formatierung (im Gegensatz zu Word), ohne die Komplexität von LaTeX. Der Hauptvorteil von plantuml war, dass ich die Diagramme als code abspeichern konnte. Dadurch konnte ich die Diagramme einerseits sehr einfach anpassen und andererseits einfacher als Kontext an KI weitergeben.

Die Verwendung von KI war sehr hilfreich beim Lernen der neuen Programmiersprache. Ich konnte Anfängerfehler direkt verbessern und Designentscheidungen welche mir später in den Weg kommen würden, konnten frühzeitig erkannt und verbessert werden. Ausserdem konnte mich die KI daran erinnern, wenn ich beim Implementieren Abweichungen zum Konzept hatte.

6. Anhang

6.1. Parktarife

Die nachfolgenden Tarife sind dem Lastenheft entnommen und bilden die Grundlage für die Implementierung der Gebührenberechnung.

Zeitraum	Tarif
Wochentage	
00:00 - 05:59	CHF 2.50 / Std.
06:00 - 08:59	CHF 2.80 / Std.
09:00 - 17:59	CHF 3.60 / Std.
18:00 - 20:59	CHF 2.80 / Std.
21:00 - 23:59	CHF 2.40 / Std.
Wochenende und Feiertage	
00:00 - 08:59	CHF 2.40 / Std.
09:00 - 17:59	CHF 3.20 / Std.
18:00 - 23:59	CHF 2.40 / Std.

Tabelle 47: Parktarife (Quelle: Lastenheft)

Die Abrechnung erfolgt auf Viertelstundenbasis. Der Tarif zu Beginn einer Viertelstunde gilt für die gesamte Viertelstunde. Bei einer Parkdauer von über 24 Stunden wird eine Tagespauschale von CHF 35.00 pro Tag angewendet. Angebrochene Tage werden vollständig verrechnet.

6.2. Lastenheft

Das Lastenheft ist als separater Anhang verfügbar.

Literaturverzeichnis

Tabellenverzeichnis

Tabelle 1	Abkürzungsverzeichnis	1
Tabelle 2	Stakeholderanalyse	2
Tabelle 3	Variantenvergleich	3
Tabelle 4	Variantenvergleich Webframework	3
Tabelle 5	Variantenvergleich Vorgehensmodell	4
Tabelle 6	Risikoanalyse	5
Tabelle 7	Funktionale Anforderungen	9
Tabelle 8	Nicht-funktionale Anforderungen	9
Tabelle 9	Organisatorische Anforderungen	10
Tabelle 10	Use Case UC-01	10
Tabelle 11	Use Case UC-02	10
Tabelle 12	Use Case UC-03	11
Tabelle 13	Use Case UC-04	11
Tabelle 14	Use Case UC-05	12
Tabelle 15	Use Case UC-06	12
Tabelle 16	Use Case UC-07	13
Tabelle 17	Use Case UC-08	13
Tabelle 18	Use Case UC-09	14
Tabelle 19	Use Case UC-10	14
Tabelle 20	Use Case UC-11	15
Tabelle 21	Use Case UC-12	15
Tabelle 22	Use Case UC-13	16
Tabelle 23	Testfälle	30
Tabelle 24	Datenbankstruktur	39
Tabelle 25	Testergebnis TC-01	41
Tabelle 26	Testergebnis TC-02	42
Tabelle 27	Testergebnis TC-03	42
Tabelle 28	Testergebnis TC-04	42
Tabelle 29	Testergebnis TC-05	42
Tabelle 30	Testergebnis TC-06	43
Tabelle 31	Testergebnis TC-07	43
Tabelle 32	Testergebnis TC-08	43
Tabelle 33	Testergebnis TC-09	43
Tabelle 34	Testergebnis TC-10	44
Tabelle 35	Testergebnis TC-11	44
Tabelle 36	Testergebnis TC-12	44
Tabelle 37	Testergebnis TC-13	45

Tabelle 38	Testergebnis TC-14	45
Tabelle 39	Testergebnis TC-15	45
Tabelle 40	Testergebnis TC-16	45
Tabelle 41	Testergebnis TC-17	46
Tabelle 42	Testergebnis TC-18	46
Tabelle 43	Testergebnis TC-19	46
Tabelle 44	Testergebnis TC-20	46
Tabelle 45	Testergebnis TC-21	47
Tabelle 46	Testergebnis TC-22	47
Tabelle 47	Parktarife (Quelle: Lastenheft)	48
Tabelle 48	Hilfsmittel (eigene Darstellung)	51
Tabelle 49	Glossar (eigene Darstellung)	51

Abbildungsverzeichnis

Abbildung 1	Zeitplan	5
Abbildung 2	Kontextdiagramm	7
Abbildung 3	Geschäftsprozess	8
Abbildung 4	Ausfahrt	17
Abbildung 5	Bezahlung	18
Abbildung 6	Einfahrt Dauermieter	19
Abbildung 7	Einfahrt Gelegenheitsnutzer	20
Abbildung 8	Statistiken (Umsatz)	21
Abbildung 9	Klassendiagramm	22
Abbildung 10	Parkplatz Zustandsdiagramm	25
Abbildung 11	Ticket Zustandsdiagramm	26
Abbildung 12	ERD	27
Abbildung 13	Architekturdiagramm	29
Abbildung 14	Modulübersicht	34
Abbildung 15	Einfahrt Gelegenheitsnutzer	36
Abbildung 16	Dauermieter-Bereich	37
Abbildung 17	Admin Login	37
Abbildung 18	Admin Ansicht	38
Abbildung 19	Statistik-Ansicht	38
Abbildung 20	Etagen Ansicht	39
Abbildung 21	Entity-Relationship-Diagramm	40

Hilfsmittelverzeichnis

Welches Hilfsmittel wurde eingesetzt?	Wozu wurde das Hilfsmittel eingesetzt?	Betroffene Stellen
Typst	Dokumentenerstellung, Rechtschreibkorrektur	Gesamtes Dokument
Claude	Dokumentation abgleichen mit Code, Code generierung	Initialisierung, Konzept, Realisierung
plantuml	Diagramm Erstellung	Abbildungen

Tabelle 48: Hilfsmittel (eigene Darstellung)

Glossar

Fachwort	Bedeutung
BEAM (Bogdan's Erlang Abstract Machine)	Die virtuelle Maschine, auf der Elixir und Erlang ausgeführt werden. Sie unterstützt Nebenläufigkeit und hohe Fehlertoleranz.
DSL (Domain Specific Language)	Eine auf einen bestimmten Anwendungsbereich zugeschnittene Abfragesprache. Im Projekt wird die Ecto-Query-DSL für Datenbankabfragen verwendet.
UUID (Universally Unique Identifier)	128-Bit-Bezeichner, der weltweit eindeutig ist. Im Projekt als Primärschlüssel für Parktickets verwendet.

Tabelle 49: Glossar (eigene Darstellung)

Eigenständigkeitserklärung

Hiermit bestätige ich, dass ich die vorliegende Semesterarbeit selbstständig erstellt habe und nur die angegebenen Quellen verwendet wurden.

Bern, 11. Mai 2026

Adrian Aeschlimann